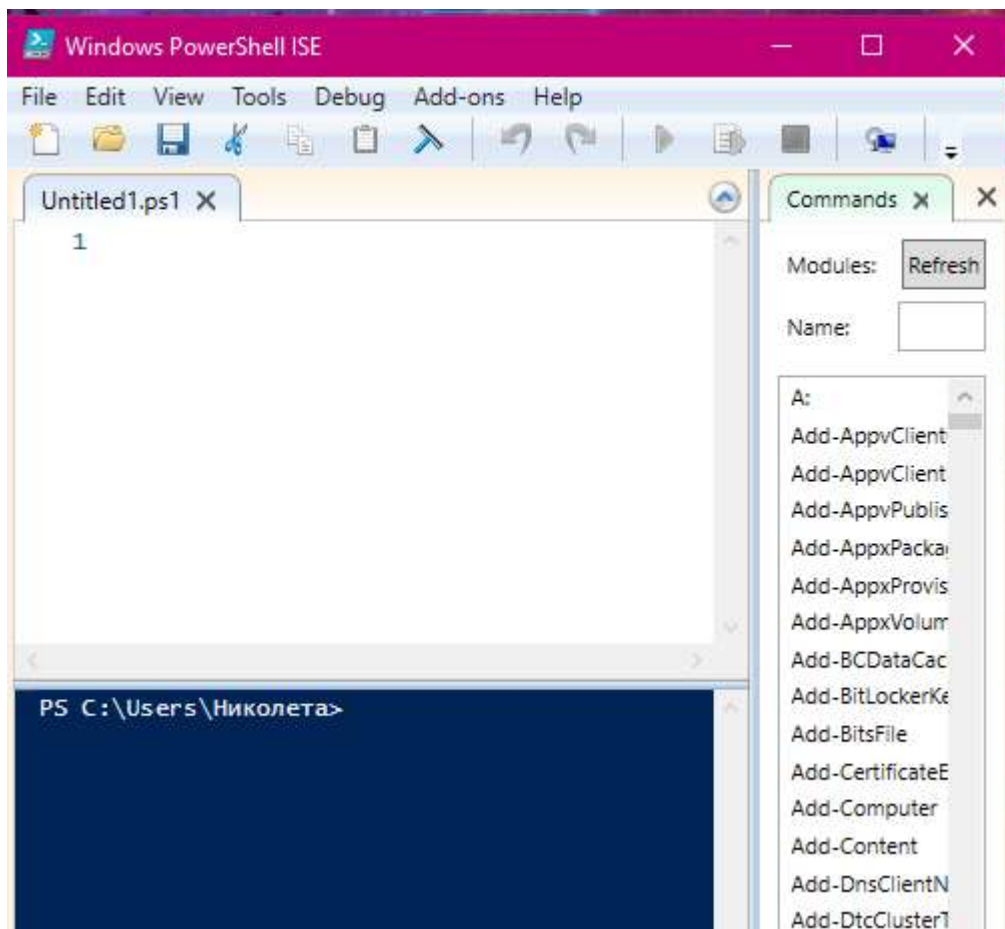
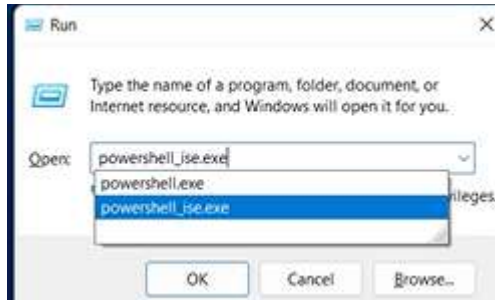


## POWERSHELL

ISE = integrated scripted environment – можем да пишем свои скриптове

2 modes = CLI + ISE



\$ = string character

```
PS C:\Users\Николета> $psversiontable.psversion

Major Minor Build Revision
-----
5      1      19041  3031
```

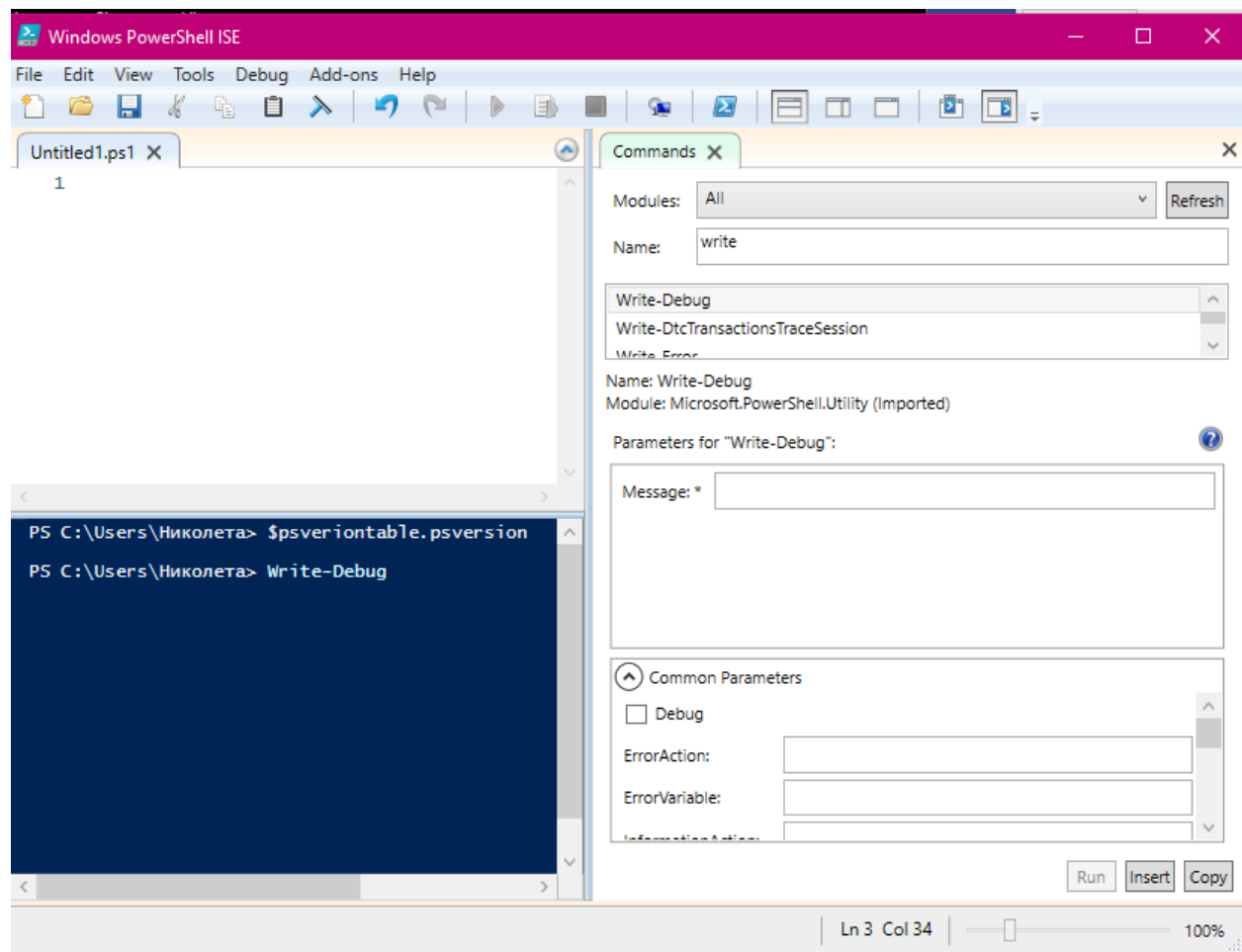
\$psversiontable.psversion – проверяваме версията

Можем remote да въдейнем едновременно много машини.

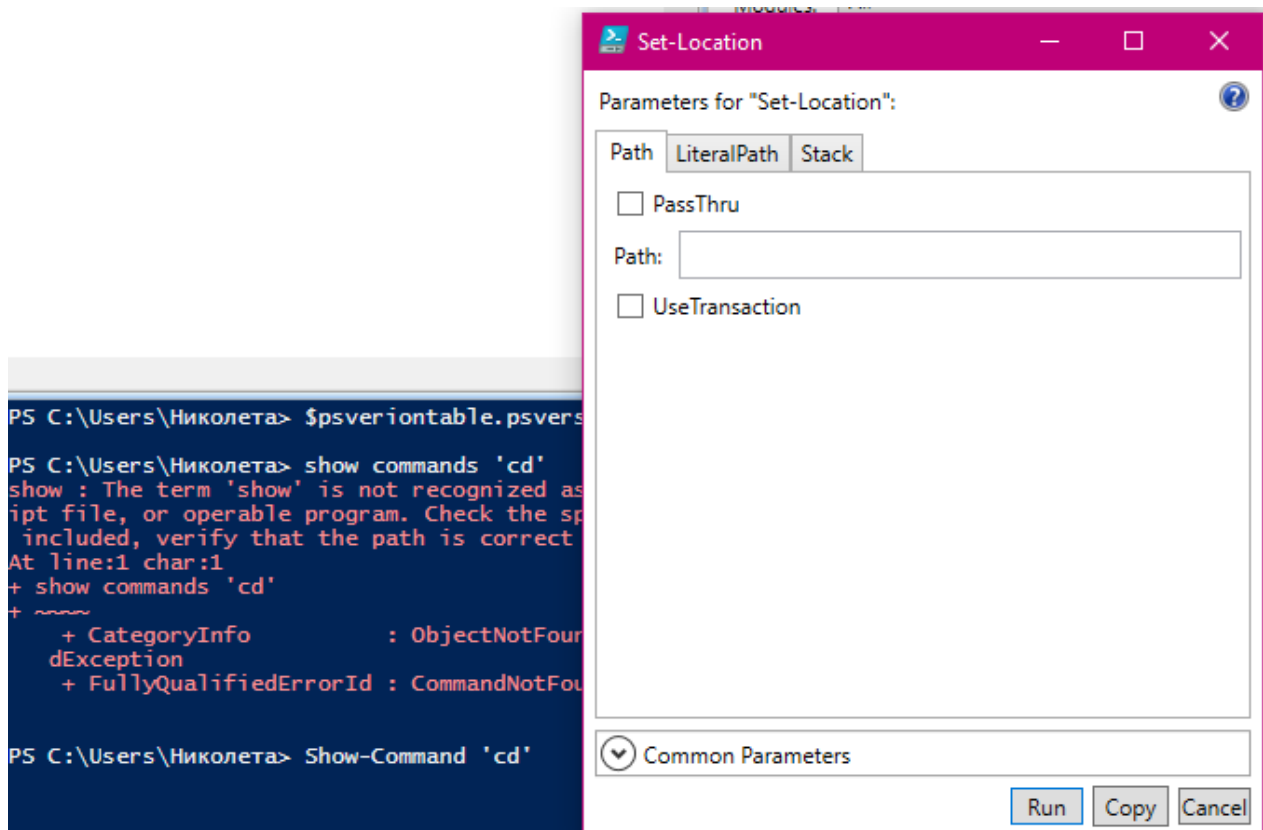
Можем да изгледим windows management framework 5.1

В ISE можем да разглеждаме съдържанието на текущата директория с `ls = list`

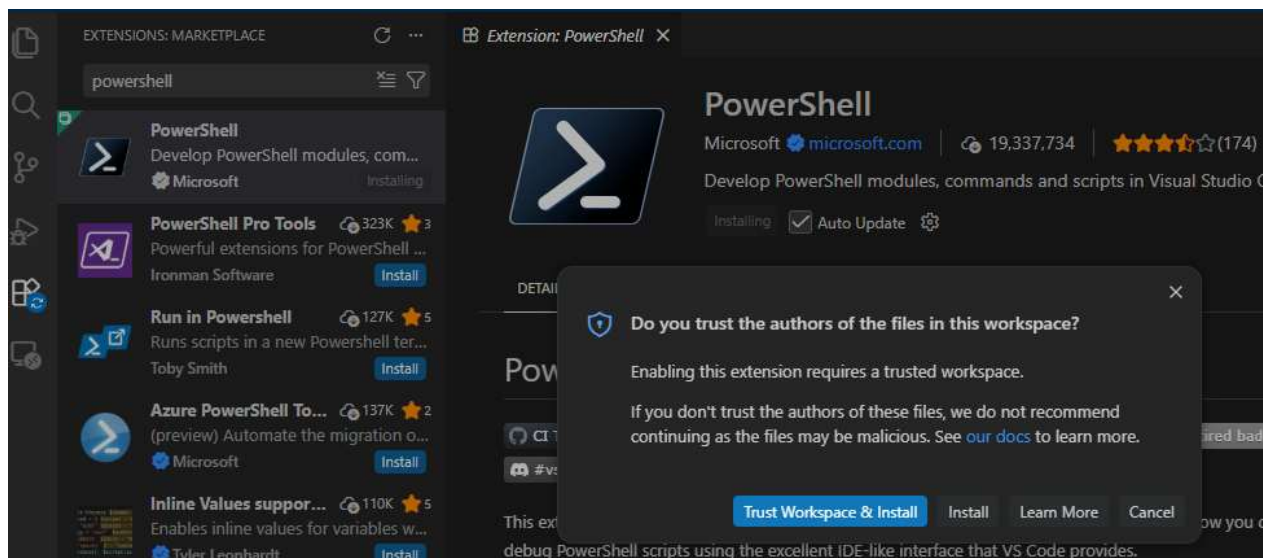
Десния команден панел се използва като търсачка или по модул, или по дума. Можем да подадем параметри и от бутон Insert да вмъкнем командата



Show-Command 'cd' – показва параметрите, които могат да се подадат



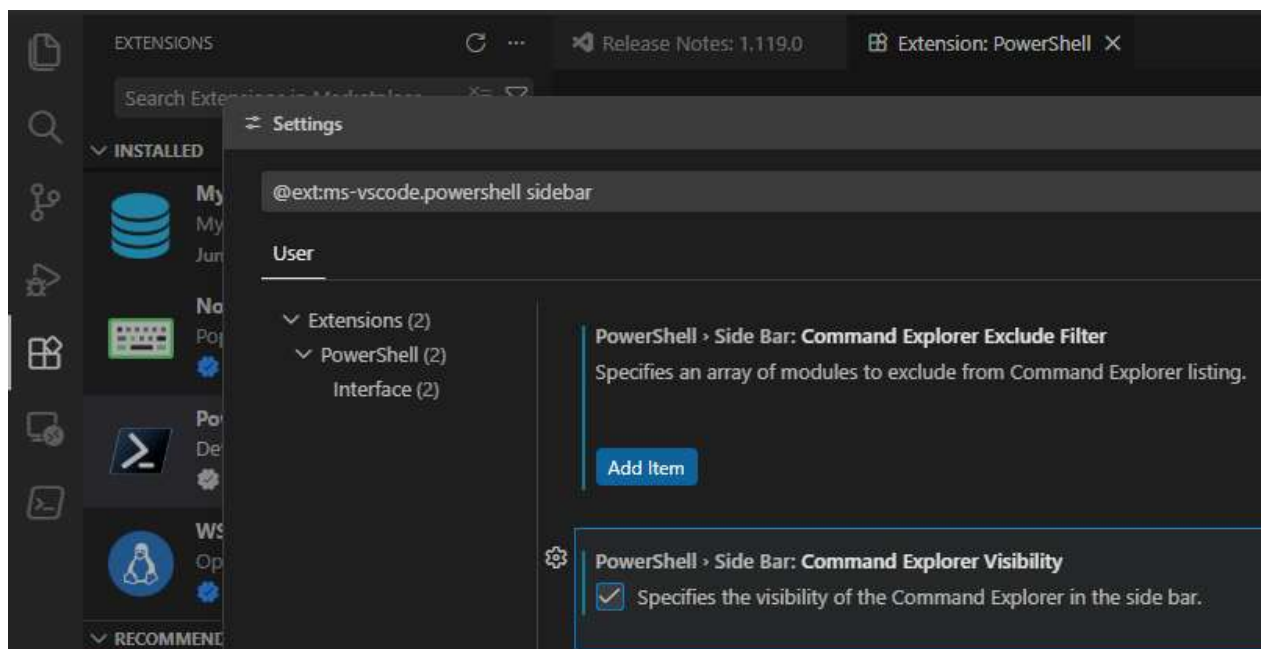
PowerShell in VisualStudio Code – инсталираме приставка



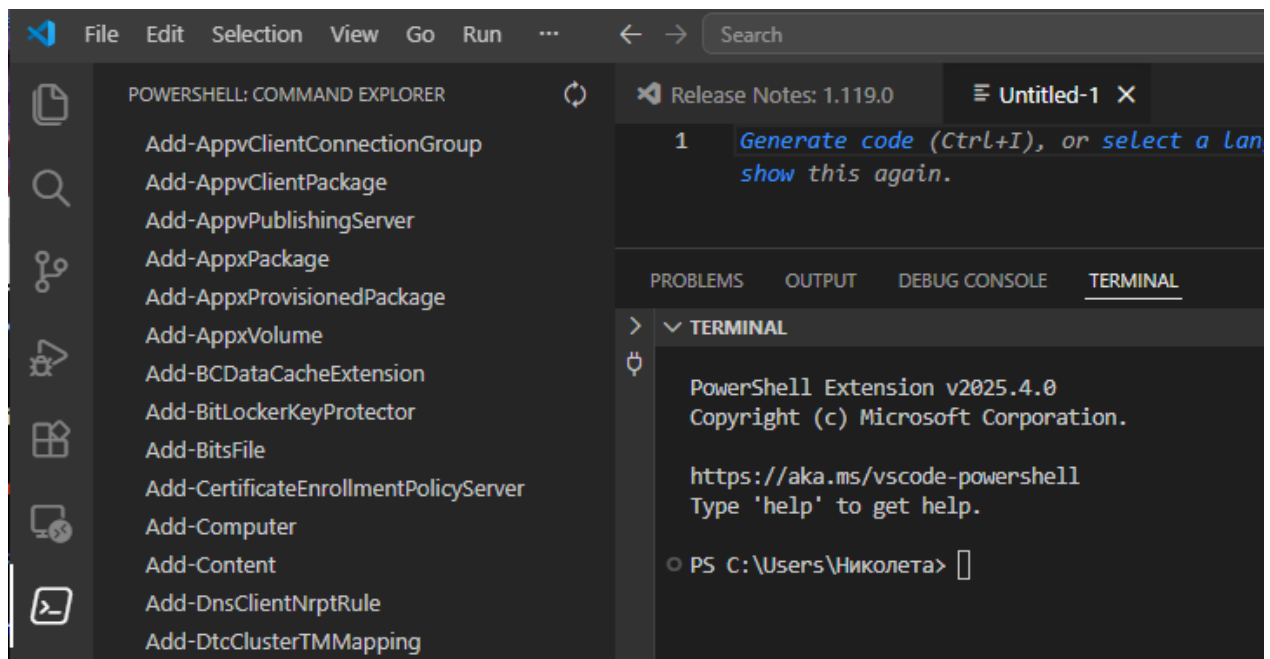
Прави същото като CMD, локално

```
NSOLE  TERMINAL
▼ TERMINAL
PS C:\Users\Николета> ls
-a----      25.2.2025 г.      0:43      8388608 test_log.ldf
-a----      22.4.2025 г.      7:40      8388608 WebApplication11
                        .mdf
-a----      22.4.2025 г.      7:40      8388608 WebApplication11
                        _log.ldf
-a----      15.6.2025 г.      2:26          1199 _viminfo
```

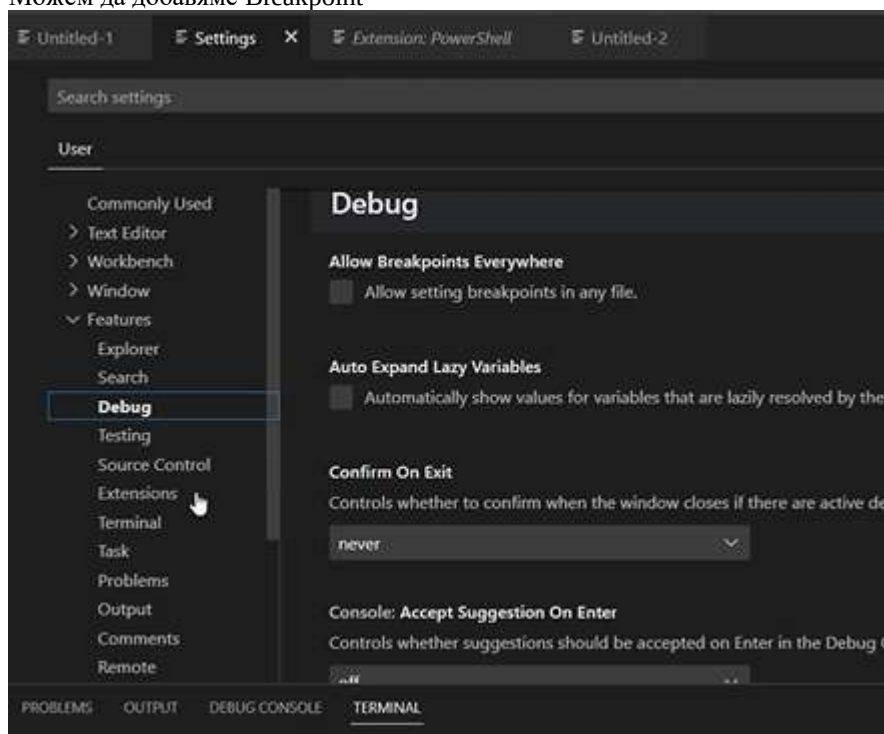
Правим добавката видима в страничния панел



Вляво можем да търсим команди



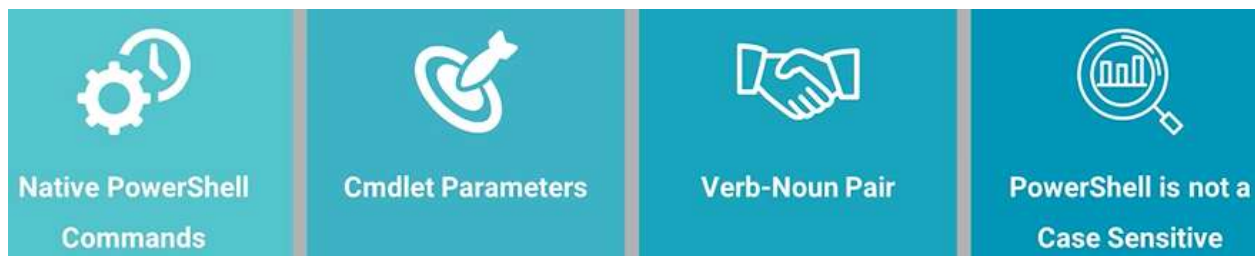
Можем да добавяме Breakpoint



Powershell = OOP, не е Key Sensitive!!

Commandlet = не се използват самостоятелно

Verb – noun pair = като move – item, new – item...



Командите са специални класове .net, написани на високо ниво езици и се компилират в .dll, за да бъдат изпълнени. Могат да бъдат извикани по време на Runtime от API. Commandlets могат да имат променливи и стойности, които да бъдат прикачени. Могат да бъдат задължителни или не.

Write-host – написва съобщение, commandlet + value = “hello”

```
PS C:\Users\Николета> write-host "hello"
hello
```

Може да се използва Tab за дописване, подреждането е по азбучен ред. Със стрелка нагоре използваме команди от сесията.

-ForegroundColor без параметър връща грешка.

```
PS C:\Users\Николета> write-host "hello" -ForegroundColor
Write-Host : Missing an argument for parameter 'ForegroundColor'.
try again.
At line:1 char:20
+ write-host "hello" -ForegroundColor
+ ~~~~~
+ CategoryInfo          : InvalidArgument: (:) [Write-Host],
+ FullyQualifiedErrorId : MissingArgument,Microsoft.PowerShell
PS C:\Users\Николета> write-host "hello" -ForegroundColor green
hello
```

get-command – изписва всички команди, а списъкът е огромен

get-command write-\* - изписва всички, които започват с write

get-command \*server – всички команди, които завършват със server

get-help \*msol\* - намира всички, които съдържат msol

get-help update-azrouteserver – връща пълно описание и синтаксис на параметрите

clear – почиства екрана

get-location – къде се намираме

cd .. = връщаме се 1 директория нагоре

new-item -name [име] – създаваме нова папка

remove-item [име]

copy-item

move-item

rename-item

```
PS C:\Users\Николета> new-item -name testfolder

Directory: C:\Users\Николета

Mode                LastWriteTime         Length Name
----                -
-a-----         11.5.2026 г.         16:15           0 testfolder
```

Get-process – колко много процеси изписва

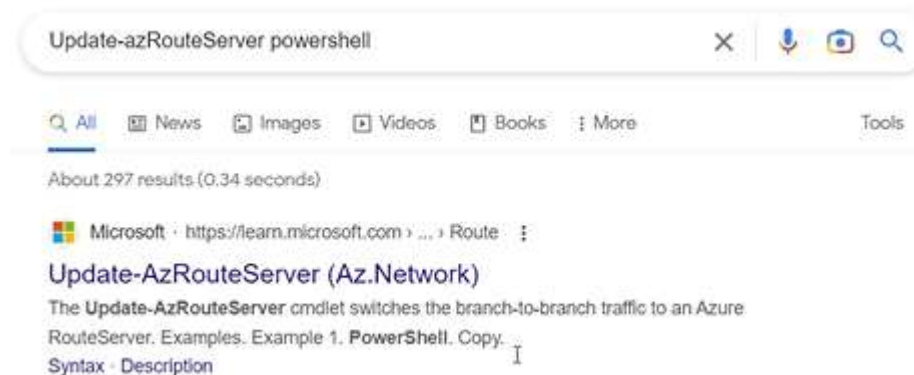
Get-process [име] – показва конкретен процес (+ id), а ако объркаме името, ще изведе грешка

Stop-process -id [номер]

Gpupdate /force = най-често използваната команда, ъпдейтва

```
PS C:\Users\Николета> gpupdate /force
Updating policy...

Computer Policy update has completed successfully.
```



Ctrl + F търсим конкретен параметър

## Parameters

### -AllowBranchToBranchTraffic

Flag to allow branch to branch traffic for route server.

|                             |                 |
|-----------------------------|-----------------|
| Type:                       | SwitchParameter |
| Position:                   | Named           |
| Default value:              | None            |
| Accept pipeline input:      | False           |
| Accept wildcard characters: | False           |

Ако напишем само командата като Update-AzRouteServer без задължителни параметри, PW ще ни поиска да я допишем, но ние трябва да напишем още веднъж целия синтаксис с нея включително, а не само тя, например Update-AzRouteServer -ResourceGroupName testgroup -RouteServerName testserver

Имаме и позиционни и непозиционни параметри

Get-Help Get-EventLog – можем да потърсим и информация за конкретен параметър от командата като Before

```
PS C:\Users\Николета> Get-Help Get-EventLog -Parameter Before
-Before <datetime>

Required?           false
Position?          Named
Accept pipeline input? false
Parameter set name  LogName
Aliases            None
Dynamic?           false
```

Update-Help – ъпдейтваме help менюто, което е желателно периодично, но отнема време

## Exercise 01: Executing and Analyzing PowerShell Commands in Windows 10 Environment

- \* PowerShell ISE: Click on the **New Script** icon which is shown in the image. A new script page appears where you can type a script.
- \* To check the PowerShell version, type `$psversiontable.psversion` and press **Enter**.
- \* You can see in the table the **Major** value is **5** and **Minor** value is **1**. Thus, the version of PowerShell is 5.1. The advantage of the 5.1 version is backward compatibility which means all actions as operations can be performed without any changes.
- \* If the PowerShell version is below 5.1, then one can download it via Google. Minimize the **Windows PowerShell**. Click on the **Google Chrome** icon on the desktop.
- \* Google Chrome opens. If the **What's New** tab is open, then click on **New Tab**.
- \* In the URL search bar, type windows management framework 5.1 and press **Enter**. The search result appears.
- \* Click on the **Microsoft** link in your page, to open the official website to download framework version 5.1.
- \* The Download tab appears as shown. Scroll down to **Windows Management Framework 5.1**.
- \* Scroll down and click on **System Requirements**. It gives the list of machines in which the framework can be downloaded. You can see in the **IMPORTANT** section that WMF 5.1 requires Microsoft.NET Framework 4.5 or above.

## Exercise 02: Launching and Navigating Windows PowerShell ISE: Exploring Command Modules and Installing PowerShell in Visual Studio Code

- \* Type **ls** in the PowerShell console pane and press **Enter**. The result gives the list of files in your current working directory.
- \* On the right-hand side of the PowerShell console, you can see the **Commands** window. It shows all the available commands. If the **Commands** section is not visible, then click on **Show Command Add-on** which is present at the last of the toolbar.
- \* In the **Modules**, click on scroll down icon. Click on any available module. Here **AppvClient** module is selected. It gives the list of commands associated with that module.
- \* Check for the commands of other modules. Click on **downs scroll** again. Here **AppLocker** module is selected. You can see the list of commands available for that module.
- \* Now, set it back to default, by selecting **All** modules.

- \* Type **write** in the **Name** section. You can see the list of commands having the word **write** in it. You can use the **Name** section when you don't know with which specific module the command is associated.
- \* Click on **Write-Debug**. You will get additional information about it. Click on **Common Parameters** present at the bottom. You can see various parameters you can use.
- \* At the bottom click on the **Insert** button as shown in the image. You can see in the PowerShell console pane that **Write-Debug** is typed automatically.
- \* Remove the **Write-Debug** in console. Click on the **py** button at the right bottom of the page, as shown in the image.
- \* Move the cursor to the console section and give it a click. Press **Ctrl+V** to paste it. You can see that **Write-Debug** is pasted.
- \* Remove it and type **cd** as shown in the image. Do not press Enter.
- \* Click on the **Show Command Window** tool which is shown in the image. You can see a pop-up with available parameters with check boxes of that command.
- \* In the parameter pop-up, you can see **Run**, **Copy**, and **Cancel** buttons are available. Click on **Cancel** as you will not be running or copying the command.
- \* Click on the **Show Command Window** tool which is shown in the image. You can see a pop-up with available parameters with check boxes of that command.
- \* In the parameter pop-up, you can see **Run**, **Copy**, and **Cancel** buttons are available. Click on **Cancel** as you will not be running or copying the command.
- \* Minimize the **Windows PowerShell**. Click on the **Google Chrome** icon on the desktop.
- \* Now, you will learn how to install Powershell in Visual Studio Code. For that first, you must download Visual Studio Code. So, type visual studio code in the Google search bar and press **Enter**.
- \* Click on the **Extensions** icon. In the **Search Extensions in Marketplace**, type **powershell**. Then click on the **Install** of the first option. (...)

## Exercise 03: Executing Basic PowerShell Commands in Windows 10

- \* Type write-host "hello" and press **Enter**. You can see the output result as **hello**. **write-host** allows you to show the output to the information stream.
- \* Now, just type write-h and press **Tab**. You can see the command is autocompleted, but it is not executed. Append it with the "first try" input value. The entire command would be Write-Host "first try". Then press **Enter**. It gives the output as shown.
- \* Now, just type **w**. Then press **Tab**. Each time you press **Tab**, the PowerShell will suggest different commandlets starting from "w".
- \* Delete the command. Press **Upper arrow** from the keyboard. You can see it is auto-filled with the last executed command. Press **Enter** to execute it.
- \* Type Write-Host "first try" -f. Then press **Tab**. The entire command will be **Write-Host "first try" -ForegroundColor**. Then press **Enter**.
- \* Instead of output, you will get an error. That is because **ForegroundColor** is a parameter to which you must give an argument.
- \* Now, type Write-Host "first try" -ForegroundColor green and press **Enter**. You can see the output is green color.
- \* Type get-command and press **Enter**. You will get a list of PowerShell commands that are pre-installed. The list has the name of the command, its type, version, and source.

- \* As you can see the page is overfilled, type **clear** and press **Enter**. You can see that the page is cleared.
- \* Now, type **get-location** and press **Enter**. You can see the current directory of the current PowerShell run space.
- \* To get the list of files and directories in your current directory, type **ls** and press **Enter**.
- \* To change the directory path type **cd ..** and press **Enter**. You can see the path has changed.
- \* Type **ls** and press **Enter** to see the content of the current directory.
- \* To move back to the older directory path, type **cd Admin** and press **Enter**.
- \* To add a new item to the current directory, type **new-item -name NEWITEM** and press **Enter**. **new-item** is used to create a new item and set its value. Here the value is **NEWITEM**.
- \* Type **ls** and press **Enter**. You can see the **NEWITEM** is added.
- \* To remove the item, type **remove-item NEWITEM** and press **Enter**.
- \* Type **ls** and press **Enter**. You can see the item is deleted from the list.
- \* Type **get-process** and press **Enter**. **get-process** is used to get the processes on a local or remote computer.
- \* Type **get-process wordpad** and press **Enter**. You will get an error as the CPU is not yet allotted to it.
- \* Minimize the **Windows PowerShell**. Click on the **Search** icon in the toolbar. Type **word**. Click on **WordPad**.
- \* Once the **WordPad** is opened, the processor will be allotted to it in the back end. Minimize the **WordPad**. Re-open the **Windows PowerShell** from the desktop toolbar.
- \* Press the **Upper arrow** from the keyboard to autofill the previously executed command. Then press **Enter**. You can see the output as shown. See the **Id** value assigned to wordpad is **2340**. It might vary in your lab. You can also see the **WordPad** icon in the toolbar as it is running.
- \* Type **stop-process -Id 2340** and press **Enter**. The command will stop the process for **WordPad**. You can see that Wordpad has been removed from the taskbar. Type the **Id** value that you got in your lab.

## Exercise 04: Explore and Identify PowerShell Commands and Their Parameters

- \* Type **get-command** and press **Enter**. As a result, you will get a list of commandlets of PowerShell.
- \* To get the list of commandlets that start with **write**. Type the following command and press **Enter**. **get-command write-\***
- \* **get-help \*client\***
- \* **get-command \*server**
- \* Type **get-help Update-AutologgerConfig** and press **Enter**. You can see in the **SYNTAX** section has two values given. It means that the command can be used in two ways with the parameters as given in syntax. The parameters in square brackets indicate that they are optional.
- \* If one does not know the parameters used in the syntax, one can understand it via Google search. Minimize the **Windows PowerShell** and click on **Google Chrome** on the desktop.

ISE:

- \* Type **update-auto**, you can see that the **Update-AutologgerConfig** command is recommended.

Type **get-help get-eventlog** and press **Enter**. The information about **get-eventlog** will be shown.

If you want to get information about the parameter of the command within PowerShell, use **-parameter**. To check it, type **get-help get-eventlog -parameter before** and press **Enter**.

## Remote Server Management

PowerShell Remote should be enabled

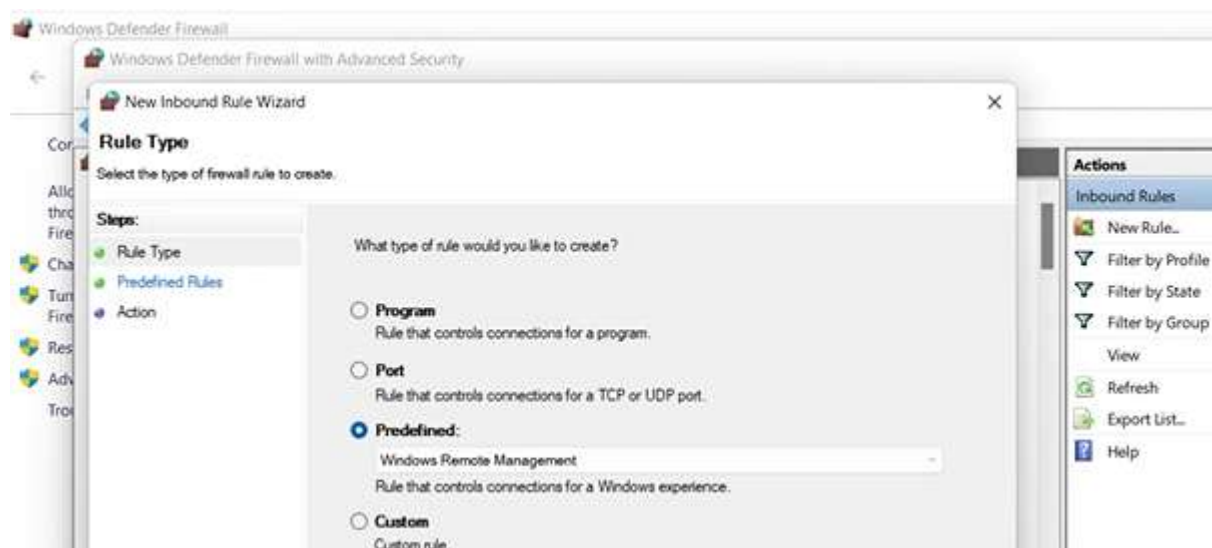
```
PS C:\Users\Николета> Enter-PSsession -ComputerName DESKTOP-T4V50B3
Enter-PSsession : Connecting to remote server DESKTOP-T4V50B3 failed with the following error message : The client cannot connect to the
destination specified in the request. Verify that the service on the destination is running and is accepting requests. Consult the logs
and documentation for the WS-Management service running on the destination, most commonly IIS or WinRM. If the destination is the WinRM
service, run the following command on the destination to analyze and configure the WinRM service: "winrm quickconfig". For more informa
tion, see the about_Remote_Troubleshooting Help topic.
At line:1 char:1
+ Enter-PSsession -ComputerName DESKTOP-T4V50B3
+ ~~~~~
+ CategoryInfo          : InvalidArgument: (DESKTOP-T4V50B3:String) [Enter-PSsession], PSRemotingTransportException
+ FullyQualifiedErrorId : CreateRemoteRunspaceFailed
```

```
PS C:\Users\jsimunek1> Enter-PSsession -ComputerName localhost
[localhost]: PS C:\Users\jsimunek1\Documents> exit
PS C:\Users\jsimunek1>
```

+ exit of the session

```
PS C:\Users\Николета> enable-PSremoting
enable-PSremoting : Access is denied. To run this cmdlet, start Windows PowerShell with the "Run as administrator" option.
At line:1 char:1
+ enable-PSremoting
+ ~~~~~
+ CategoryInfo          : NotSpecified: (:) [Enable-PSremoting], InvalidOperationException
+ FullyQualifiedErrorId : System.InvalidOperationException,Microsoft.PowerShell.Commands.EnablePSRemotingCommand
```

Windows Defender Firewall -> Advanced -> Inbound Rules -> New Rule -> Windows Remote Management, Port 5985



Group Policy – може да включим remote management на много машини наведнъж.  
Обикновено, remote е 1:1, а не 1:n.

Remote WS to Linux/Mac -> edit system wide configuration file /etc/ssh/sshd\_config + edit /user/bin/pwsh

Notepad.exe може да се отвори като нормална програма с GUI локално, но ако се опитаме да я отворим по време на remote сесия, не отговаря, защото терминала я отваря, но виси без интерфейс.

```
PS C:\Users\jsimunc1> notepad.exe
PS C:\Users\jsimunc1> Enter-PSSession -ComputerName localhost
[localhost]: PS C:\Users\jsimunc1\Documents> notepad.exe
```

Exit 2 пъти – излиза от сесия + затваря терминала. Имаме и по-конкретен Exit-PSSession

```
PS C:\Users\jsimunc1> Enter-PSSession
cmdlet Enter-PSSession at command pipeline position 1
Supply values for the following parameters:
ComputerName: localhost
[localhost]: PS C:\Users\jsimunc1\Documents> Exit-PSSession
PS C:\Users\jsimunc1> exit
```

```
PS C:\Users\jsimunc1> Get-PSSession
PS C:\Users\jsimunc1> New-PSSession
```

| Id | Name   | ComputerName | ComputerType  | State  | ConfigurationName    | Availability |
|----|--------|--------------|---------------|--------|----------------------|--------------|
| 1  | WinRM1 | localhost    | RemoteMachine | Opened | Microsoft.PowerShell | Available    |

```
PS C:\Users\jsimunc1> Get-PSSession
```

| Id | Name   | ComputerName | ComputerType  | State  | ConfigurationName    | Availability |
|----|--------|--------------|---------------|--------|----------------------|--------------|
| 1  | WinRM1 | localhost    | RemoteMachine | Opened | Microsoft.PowerShell | Available    |

```
PS C:\Users\jsimunc1> New-PSSession
```

| Id | Name   | ComputerName | ComputerType  | State  | ConfigurationName    | Availability |
|----|--------|--------------|---------------|--------|----------------------|--------------|
| 2  | WinRM2 | localhost    | RemoteMachine | Opened | Microsoft.PowerShell | Available    |

Създаваме нови сесии с new-pssession, проверяваме с get-pssession. При затваряне на терминала, всички сесии се затварят, но има и друга команда: Disconnect-PSSession –id 3 (спира конкретно 3-та сесия). Включваме с Connect-PSSession –id 3

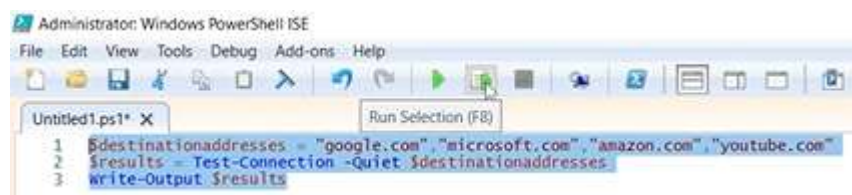
Remote Tasks to perform

Test-connection google.com –Quiet

```
[localhost]: PS C:\Users\jsimunc1\Documents> Test-Connection google.com -Quiet
True
[localhost]: PS C:\Users\jsimunc1\Documents> Test-Connection microsoft.com -Quiet
False
[localhost]: PS C:\Users\jsimunc1\Documents> Test-Connection amazon.com -Quiet
True
[localhost]: PS C:\Users\jsimunc1\Documents> Test-Connection youtube.com -Quiet
True
```

Result true/false

Автоматизиране на скрипт:



```
[localhost]: PS C:\Users\jsimunc1\Documents> $destinationaddresses = "google.com","microsoft.com","amazon.com","youtube.com"
$results = Test-Connection -Quiet $destinationaddresses
Write-Output $results
True
False
True
True
```

За над 100 адреса, този скрипт е полезен.

“random text” > testfile.txt – създаваме нов текстов файл със съдържание

\$var = New-PSSession localhost – създаваме нова локална сесия

Копираме файла върху локалния сървър в C:\

```
PS C:\Users\jsimunc1> Copy-Item testfile.txt -ToSession $var -Destination c:\
PS C:\Users\jsimunc1> Enter-PSSession -ComputerName localhost
[localhost]: PS C:\Users\jsimunc1\Documents> cd ..
[localhost]: PS C:\Users\jsimunc1> cd ..
[localhost]: PS C:\Users> cd ..
[localhost]: PS C:\> ls
```

Могат да се конвертират PS outputs -> excel files.

```
PS C:\Users\jsimunc1> Get-PSSession localhost
```

| Id | Name   | ComputerName | ComputerType  | State        | ConfigurationName    | Availability |
|----|--------|--------------|---------------|--------------|----------------------|--------------|
| 2  | WinRM2 | localhost    | RemoteMachine | Opened       | Microsoft.PowerShell | Available    |
| 1  | WinRM1 | localhost    | RemoteMachine | Opened       | Microsoft.PowerShell | Available    |
| 4  | WinRM1 | localhost    | RemoteMachine | Disconnected | Microsoft.PowerShell | None         |
| 5  | WinRM3 | localhost    | RemoteMachine | Disconnected | Microsoft.PowerShell | None         |
| 6  | WinRM2 | localhost    | RemoteMachine | Disconnected | Microsoft.PowerShell | None         |
| 7  | WinRM1 | localhost    | RemoteMachine | Disconnected | Microsoft.PowerShell | None         |

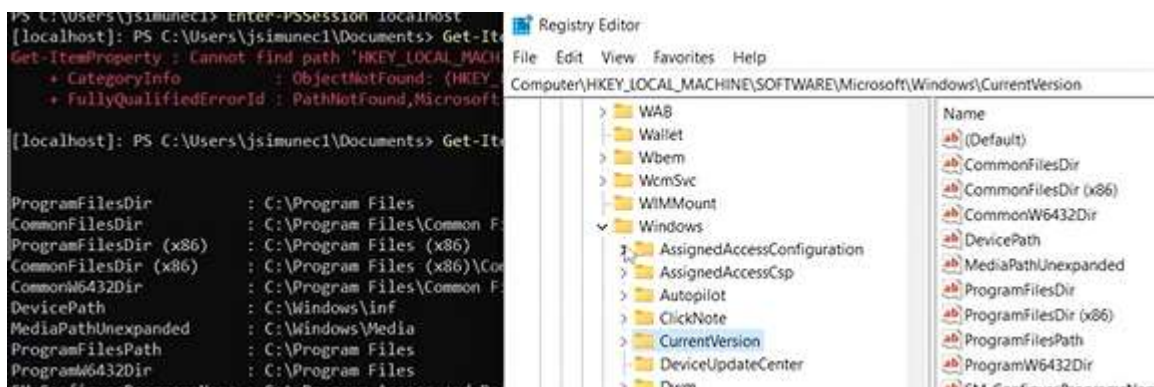
```
PS C:\Users\jsimunc1> Enter-PSSession localhost
```

Бъркаме в регистъра:

```
[localhost]: PS C:\Users\jsimunc1\Documents> Get-ItemProperty -Path Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion

ProgramFilesDir       : C:\Program Files
CommonFilesDir        : C:\Program Files\Common Files
ProgramFilesDir (x86) : C:\Program Files (x86)
CommonFilesDir (x86)  : C:\Program Files (x86)\Common Files
CommonW6432Dir        : C:\Program Files\Common Files
DevicePath            : C:\Windows\inf
MediaPathUnexpanded   : C:\Windows\Media
ProgramFilesPath       : C:\Program Files
ProgramW6432Dir        : C:\Program Files
SM_ConfigureProgramsName : Set Program Access and Defaults
SM_GamesName          : Games
PSPath               : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion
PSParentPath          : Microsoft.PowerShell.Core\Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows
PSChildName           : CurrentVersion
PSProvider            : Microsoft.PowerShell.Core\Registry
```

При сравнение с регистъра, съдържанието е еднакво



Вместо localhost, може да стои името на друга remote машина.

Script Examination using PSScriptAnalyzer

Създаваме нова папка в C:\ -> md scripts = make directory

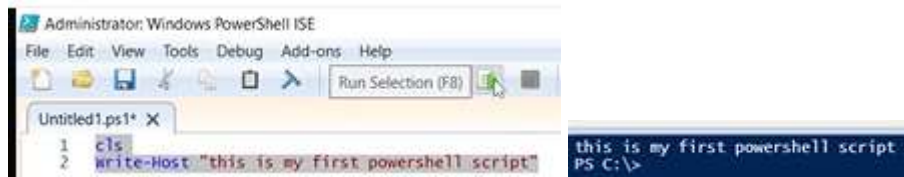
```
PS C:\> md scripts

Directory: C:\

Mode                LastWriteTime         Length Name
----                -
d-----         10.1.2023.    10:11             scripts
```

The image shows the Windows PowerShell ISE (Integrated Scripting Environment) with a script named `Untitled1.ps1` open. The script contains a single line: `Write-Host "this is my first powershell script"`. The script is being executed, and the output is displayed in the console window at the bottom: `PS C:\> Write-Host "this is my first powershell script"` followed by `this is my first powershell script`.

cls – скрива кода и показва само изхода



File->Save As -> firstscript.ps1

Може да се затвори сесията и отидем в директорията, която е скрипта и го стартираме

```
PS C:\scripts> ./firstscript.ps1
```

Install-Module -Name PSScriptAnalyzer -Force

```
PS C:\Windows\system32> Install-Module -Name PSScriptAnalyzer -Force

NuGet provider is required to continue
PowerShellGet requires NuGet provider version '2.8.5.201' or newer to interact w
provider must be available in 'C:\Program Files\PackageManagement\ProviderAssem
'C:\Users\Николета\AppData\Local\PackageManagement\ProviderAssemblies'. You can
running 'Install-PackageProvider -Name NuGet -MinimumVersion 2.8.5.201 -Force'.
and import the NuGet provider now?
[Y] Yes [N] No [S] Suspend [?] Help (default is "Y"): y
PS C:\Windows\system32>
```

-Force не е задължително, но забързва процеса

A screenshot of a PowerShell console window titled 'Administrator: Windows PowerShell'. It shows the output of a script: 'this is my first powershell script'. Below that, the command 'PS C:\scripts> Get-ScriptAnalyzerRule' is entered, followed by the command 'PS C:\scripts> Get-ScriptAnalyzerRule | Sort-Object RuleName'.

```
Administrator: Windows PowerShell
this is my first powershell script
PS C:\scripts> Get-ScriptAnalyzerRule
PS C:\scripts> Get-ScriptAnalyzerRule | Sort-Object RuleName
```

Резултатът е огромни данни

```
PS C:\scripts> Get-ScriptAnalyzerRule | Sort-Object RuleName | Select-Object CommonName
```

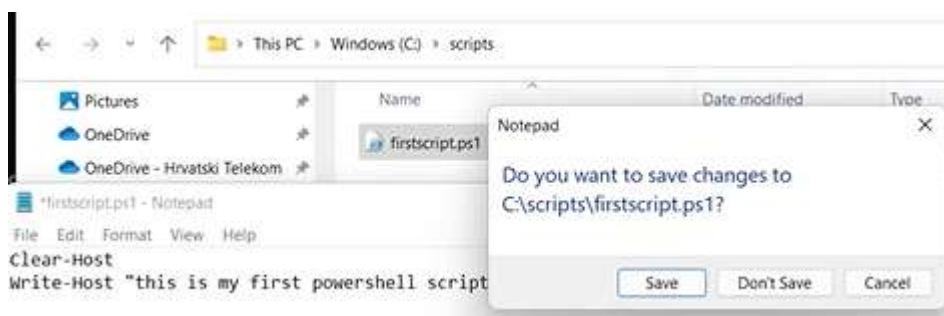
С допълнителните pipe показва по-малко данни, които са по-четими.

За да анализираме скрипт, трябва да се позиционираме в директорията на скрипта и извикаме Analyzer,

```
PS C:\scripts> Invoke-ScriptAnalyzer .\firstscript.ps1

RuleName                               Severity    ScriptName Line Message
-----
PSAvoidUsingCmdletAliases               Warning    firstscrip 1  'cls' is an alias of 'Clear-Host'. Alias can introduce possi
t.ps1                                   ble problems and make scripts hard to maintain. Please consi
der changing alias to its full content.
PSAvoidUsingWriteHost                   Warning    firstscrip 2  File 'firstscript.ps1' uses Write-Host. Avoid using Write-Host
t.ps1                                   because it might not work in all hosts, does not work whe
n there is no host, and (prior to PS 5.0) cannot be suppress
ed, captured, or redirected. Instead, use Write-Output, Writ
e-Verbose, or Write-Information.
```

Има 2 warnings, cls е по-добре да се замени с clear-host и прави същото и предупрежденията ще изчезнат.



\$ - променлива, текст в кавички, а числа директно

```
PS C:\Users\jsimunc1> $textvariable = "this is some random text"
PS C:\Users\jsimunc1> $textvariable
this is some random text
PS C:\Users\jsimunc1> $num = 12345
PS C:\Users\jsimunc1> $num
12345
```

```
Administrator: Windows PowerShell
PS C:\Windows\system32> hostname
DESKTOP-T4V5OBJ
PS C:\Windows\system32> $nam = hostname
PS C:\Windows\system32> $nam
DESKTOP-T4V5OBJ
```

Последователност:

```
PS C:\Users\jsimunc1> $arrayexample = "value1","value2","value3","value4","value5"
PS C:\Users\jsimunc1> $arrayexample
value1
value2
value3
value4
value5
```

Pipe | - мощен инструмент на последователност

```
PS C:\Users\jsimunc1> Get-Process excel | Start-Process
cmdlet Start-Process at command pipeline position 2
Supply values for the following parameters:
FilePath:
PS C:\Users\jsimunc1> Get-Process excel | Stop-Process
```

```
PS C:\Users\jsimunc1> Get-Process excel | Stop-Process
PS C:\Users\jsimunc1> function tellme { Write-Host "good morning" }
PS C:\Users\jsimunc1> tellme
good morning
```

Функциите също са много мощен инструмент, а и циклите.

```
Administrator: Windows PowerShell ISE
File Edit View Tools Debug Add-ons
Untitled1.ps1 X
1 $variablename = 2
2 Do
3 {
4 Write-Host $variablename
5 }
6 While($variablename -ne 0)
```

-ne = not equal

Безкраен цикъл, който спираме с Ctrl + C.

```
Untitled1.ps1 X
1 $totalRam = (Get-CimInstance win32_PhysicalMemory | Measure-Object -Property capacity -Sum).Sum
2 while($true) {
3 $date = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
4 $cpuTime = (Get-Counter '\Processor(_Total)\% Processor Time').CounterSamples.CookedValue
5 $availMem = (Get-Counter '\Memory\Available MBytes').CounterSamples.CookedValue
6 $date + ' > CPU: ' + $cpuTime.ToString("#,0.000") + '%, Avail. Mem.: ' + $availMem.ToString("N0") + 'MB (' + (104857600 * $availMem / $totalRam).ToString("#,0.0") + '%)'
7 Start-Sleep -s 2
8 }
```

Първо трябва да се уверим, че скриптът е безопасен и да го дебъгнем.

```
Untitled1.ps1 scriptdebugg.ps1 X
1 $totalRam = (Get-CimInstance win32_PhysicalMemory | Measure-Object -Property capacity -Sum)
2 while($true) {
3 $date = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
4 $cpuTime = (Get-Counter '\Processor(_Total)\% Processor Time').CounterSamples.CookedValue
5 $availMem = (Get-Counter '\Memory\Available MBytes').CounterSamples.CookedValue
6 $date + ' > CPU: ' + $cpuTime.ToString("#,0.000") + '%, Avail. Mem.: ' + $availMem.ToString("N0") + 'MB (' + (104857600 * $availMem / $totalRam).ToString("#,0.0") + '%)'
7 Start-Sleep -s 2
8 }
```

Run -> Step over

```
File Edit View Tools Debug Add-ons Help
Step Over F10
Step Into F11
Step Out Shift+F11
Run/Continue F5
Break All Ctrl+B
Stop Debugger Shift+F5
```

Step into + Step over изглеждат еднакви, но разликата е, че след изпълнението на текущата команда, into спира на следващата. Over също спира на следващата, но ако там е друга функция или скрипт, ще ги изпълни целите и тогава ще спре. Прилича на дебъгера на VS.

## Exercise 01: Demonstrate Secure Remote Management Using PowerShell on Windows 10

- \* Firstly, change the current location to **C:\Users\Admin**. Type **cd ..** and press **Enter**. Run the command twice.
- \* Type **cd Users** and press **Enter** to changes the current location to **C:\Users..**
- \* Type **cd Admin** and press **Enter** to change the current location to **C:\Users\Admin**. Thus, you have reached the required location.
- \* To check whether PSSession is pre-enabled, type **enter-pssession -computername localhost** and press **Enter**. You can see the error. You will get an error because it is not pre-enabled. **enter-possession** will start the session with a single remote computer. **-computername** parameter is used to specify to which remote computer you want to connect. **localhost** is the remote computer you will connect to.
- \* To enable it, type **enable-psremoting** and press **Enter**. **enable-psremoting** is a built-in command, to enable PSRemoting. It enables all session configurations by setting enables property of all session configuration's value to True.
- \* Now, press **above arrow** from the keyboard twice. The command **enter-pssession -computername localhost** will be auto-filled. Press **Enter**. This will start the session with the localhost.
- \* You can end the session by typing **exit** as shown in the image.
- \* If one wants to enable PowerShell remote server management for more machines at once, one could use Group Policy to do it. That might be found in some corporate environments behind firewalls. It is necessary to make sure that Windows Firewall is opened for Windows Remote Management on that machine. Now, minimize the **Administrator: Windows PowerShell**. Keep the cursor on the **search** icon in the taskbar.
- \* Type **firewall**. Select **Windows Defender Firewall** from the result. The **Windows Defender Firewall** application opens as shown. Click on **use recommended settings** to start the firewall. If the firewall is already on, then skip this step.
- \* Click on **Advanced settings**. The **Windows Defender Firewall with Advance settings** opens as shown. Click on **Inbound Rules**. The list of **Inbound Rules** will be shown. Under **Actions**, click on **New Rule**. A pop-up called **New Inbound Rule Wizard** appears as shown. (...)

```
$destinationaddresses = "google.com", "microsoft.com", "youtube.com"  
$result= Test-Connection -Quiet $destinationaddresses  
Write-Output $result
```

```
$var = new-pssession localhost
```

```
enter-pssession -computername localhost
```

```
get-pssession localhost
```

```
get-itemproperty -path  
Registry::HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion
```

```
cls  
Write-Host "This is my first script"
```

**Get-ExecutionPolicy** is used to get the execution policy for the current session.

Get-ScriptAnalyzerRule | Sort-Object RuleName1 - The command is used to get the list of PSScriptAnalyzer rules with details about each rule.

```
Get-ScriptAnalyzerRule | Sort-Object RuleName1 | Select-Object CommonName
```

```
Invoke-ScriptAnalyzer .\firstscript.ps1
```

```
Clear-Host  
Write-Host "This is my first script"
```

```
$textvariable = "This is a random text"
```

```
$arrayexample = "value1","value2","value3","value4"
```

```
get-process notepad | start-process
```

```
get-process notepad | stop-process
```

```
Function tellme { Write-Host "good morning" }
```

```
$variablename = 2
```

```
Do  
{  
Write-Host $variablename  
}  
While($variablename -ne 0)
```

Press **Ctrl+C** to stop it.

Select the entire script and press **F8** = Run

```
$totalRam = (Get-CimInstance Win32_PhysicalMemory | Measure-Object -Property capacity -  
Sum).Sum  
while($true) {  
$date = Get-Date -Format "yyyy-mm-dd HH:mm:ss"  
$cpuTime = (Get-Counter '\Processor(_Total)\% Processor Time').CounterSamples.CookedValue  
$availMem = (Get-Counter '\Memory\Available MBytes').CounterSamples.CookedValue
```

```
$date + ' > CPU: ' + $cpuTime.ToString("#,0.000") + '%, Avail. Mem.: ' +
$availMem.ToString("N0") + 'MB (' + (104857600 * $availMem / $totalRam).ToString("#,0.0")
+ '%)'
Start-Sleep -s 2
}
```

Press **F5** to run the script.

## Exercise 01: Apply and Verify DSC Configuration with PowerShell

**Get-DscResource** will retrieve the PowerShell DSC resources present in the system. Type **Get-DscResource** and press **Enter**.

DSC resources are the properties that can be configured on the target machine. In the output, you can see **File** at the top of the list, which means files and folders can be managed on the system. You also have **Archive** resource which means archives can be unzipped in .zip format at a particular path. You also have **Environment** resource which means system environment variables can be managed.

DSC = Desired State Configuratin чрез Push + Pull. Чрез Push конфигурираме remote компютри. Когато сме готови, минаваме към Pull – създава се master server = Pull server. Когато remote-компютри се свържат с него, той ги конфигурира, т.е. те се pull-ват от него. В реална ситуация, при много машини, първо push-ваме конфигурации за малко от тях, защото може да има и такива, които не са свързани с мрежата в дадения момент, когато админът е на разположение и иска да направи конфигурацията. После правим всички достъпни да pull-ват. При стотици и хиляди компютри, това е много по-удобно, защото трябва да конфигурираме само 1 сървър, за да pull-ват останалите, когато имат възможност, вместо ние да push-ваме всеки поотделно. Push = тест за надеждност на конфигурациите, а когато видим, че е успешна, pull за останалите.

Get-DscResource

```
PS C:\Users\jsimunc1> Get-DscResource
```

| ImplementedAs | Name                    | ModuleName                  | Version | Properties                                         |
|---------------|-------------------------|-----------------------------|---------|----------------------------------------------------|
| -----         | ----                    | -----                       | -----   | -----                                              |
| Binary        | File                    |                             |         | {DestinationPath, Attributes, Checksum, Content... |
| Binary        | SignatureValidation     |                             |         | {SignedItemType, TrustedStorePath}                 |
| PowerShell    | PackageManagement       | PackageManagement           | 1.4.8.1 | {Name, AdditionalParameters, DependsOn, Ensure...  |
| PowerShell    | PackageManagement       | PackageManagement           | 1.0.0.1 | {Name, AdditionalParameters, DependsOn, Ensure...  |
| PowerShell    | PackageManagementSource | PackageManagement           | 1.0.0.1 | {Name, ProviderName, SourceUri, DependsOn...}      |
| PowerShell    | PackageManagementSource | PackageManagement           | 1.4.8.1 | {Name, ProviderName, SourceLocation, DependsOn...} |
| PowerShell    | PSModule                | PowerShellGet               | 2.2.5   | {Name, AllowClobber, DependsOn, Ensure...}         |
| PowerShell    | PSRepository            | PowerShellGet               | 2.2.5   | {Name, DependsOn, Ensure, InstallationPolicy...}   |
| PowerShell    | Archive                 | PSDesiredStateConfiguration | 1.1     | {Destination, Path, Checksum, Credential...}       |
| PowerShell    | Environment             | PSDesiredStateConfiguration | 1.1     | {Name, DependsOn, Ensure, Path...}                 |
| PowerShell    | Group                   | PSDesiredStateConfiguration | 1.1     | {GroupName, Credential, DependsOn, Description...} |
| Composite     | GroupSet                | PSDesiredStateConfiguration | 1.1     | {DependsOn, PsDscRunAsCredential, GroupName, En... |
| Binary        | Log                     | PSDesiredStateConfiguration | 1.1     | {Message, DependsOn, PsDscRunAsCredential}         |
| PowerShell    | Package                 | PSDesiredStateConfiguration | 1.1     | {Name, Path, ProductId, Arguments...}              |
| Composite     | ProcessSet              | PSDesiredStateConfiguration | 1.1     | {DependsOn, PsDscRunAsCredential, Path, Credent... |
| PowerShell    | Registry                | PSDesiredStateConfiguration | 1.1     | {Key, ValueName, DependsOn, Ensure...}             |
| PowerShell    | Script                  | PSDesiredStateConfiguration | 1.1     | {GetScript, SetScript, TestScript, Credential...}  |

Виждаме File = файлове и папки могат да се конфигурират на тази машина. По-надолу виждаме възможност за архивиране в .zip и съответно разархивиране. Environment – системната среда може да се управлява. Log – може да се пишат съобщения за събития message events log. Може да инсталираме и деинсталираме Package, също да управляваме ключовете на регистъра без HKEY\_USERS и т.н. Сходни са и при Linux. За да ги има, първо трябва да присъстват на сървъра, а после да ги дадем на компютрите. Може да изтеглим още ресурси от MS gallery за допълнителни конфигурации.

Get-DscLocalConfigurationManager

AllowModuleOverWrite = False, т.е. на машината не може да се смени с нов.

ConfigurationMode = ApplyAndMonitor – инсталира и периодически наблюдава. Ще се създават събития в EventLog. Ако беше ApplyOnly, конфигурацията щеше да се приложи само веднъж.

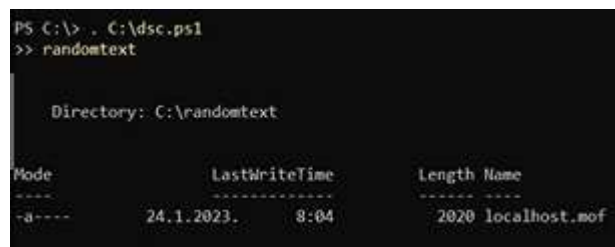
C: -> dsc.ps1:



```
File Edit Format View Help
configuration randomtext {
  Import-DscResource -ModuleName PsDesiredStateConfiguration
  Node 'localhost' {
    File randomtext {
      DestinationPath = "C:\randomtext.txt"
      Ensure = "Present"
      Contents = "this is some random text"
    }
  }
}
```

Този файл е в нашето C и ще го създадем на друга машина описанието на скрипта – файл в нейния C:\randomtext.txt, който съдържа „this is random text”. Коя машина – в нашия случай е написано Node 'localhost', но може да е на друга или други. Трябва да го конвертираме в .mof. Отиваме в дестинацията

. C:\dsc.ps1 – има точка и интервал пред C и създаваме папка randomtext. Създава се там localhost.mof



```
PS C:\> . C:\dsc.ps1
>> randomtext

Directory: C:\randomtext

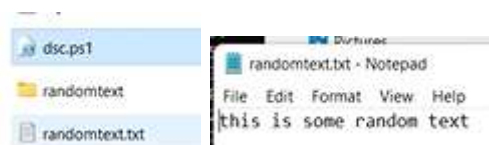
Mode                LastWriteTime         Length Name
----                -
-a----           24.1.2023.    8:04             2020 localhost.mof
```

Start-DscConfiguration –Path C:\randomtext –Verbose –Wait

```

PS C:\> Start-DscConfiguration -Path C:\randomtext -Verbose -Wait
VERBOSE: Perform operation 'Invoke CimMethod' with following parameters, 'methodName' = SendConfigurationApply, 'className' = MS
root/Microsoft/Windows/DesiredStateConfiguration'.
VERBOSE: An LCM method call arrived from computer WPAK8216 with user sid S-1-5-21-2000478354-963894560-1801674531-171919.
VERBOSE: [WPAK8216]: LCM: [ Start Set ]
VERBOSE: [WPAK8216]: LCM: [ Start Resource ] [[File]randomtext]
VERBOSE: [WPAK8216]: LCM: [ Start Test ] [[File]randomtext]
VERBOSE: [WPAK8216]: [[File]randomtext] The system cannot find the file specified.
VERBOSE: [WPAK8216]: [[File]randomtext] The related file/directory is: C:\randomtext.txt.
VERBOSE: [WPAK8216]: LCM: [ End Test ] [[File]randomtext] in 0.1120 seconds.
VERBOSE: [WPAK8216]: LCM: [ Start Set ] [[File]randomtext]
VERBOSE: [WPAK8216]: [[File]randomtext] The system cannot find the file specified.
VERBOSE: [WPAK8216]: [[File]randomtext] The related file/directory is: C:\randomtext.txt.
VERBOSE: [WPAK8216]: LCM: [ End Set ] [[File]randomtext] in 0.0060 seconds.
VERBOSE: [WPAK8216]: LCM: [ End Resource ] [[File]randomtext]
VERBOSE: [WPAK8216]: LCM: [ End Set ]
VERBOSE: [WPAK8216]: LCM: [ End Set ] in 0.6480 seconds.
VERBOSE: Operation 'Invoke CimMethod' complete.
VERBOSE: Time taken for configuration job to complete is 1.607 seconds

```



JEA = Just Enough Administration – да не се дават повече права на потребителите, отколкото са им необходими.

Session configuration file – създава environment за потребителите със специфични настройки на машината.

Get-PsSessionConfiguration – стартираме сесия, след което създаваме нов файл

New-PsSessionConfigurationFile -Path 'C:\jeaconf\file.pssc'

```
New-PsSessionConfigurationFile -Path 'C:\jeaconf\file.pssc'
```

# - коментар за ред.

```
# Session type defaults to apply for this session configuration. Can be 'RestrictedRemoteServer' (recommended), 'Empty', or 'Default'
SessionType = 'Default'
```

Променяме Default -> RestrictedRemoteServer както е препоръчително.

Нека си представим, че имаме ActiveDirectoryGroup, която се казва helpdesk и тя съдържа потребители, които трябва да приложат тази конфигурация.

```
# User roles (security groups), and the role capabilities that should be applied to them when applied to a session
# RoleDefinitions = @( 'CONTOSO\SqlAdmins' = @( RoleCapabilities = 'SqlAdministration' ); 'CONTOSO\SqlManaged' = @( RoleCapabilities = 'SqlManaged' ); 'CONTOSO\ServerMonitors' = @( VisibleCmdlets = 'Get-Process' ) )
```

За RoleCapabilities вместо SqlAdministration -> слагаме името на групата helpdesk -> Save. Създаваме нова папка RoleCapabilities -> трябва да я попълним с роли.

```
New-PSRoleCapabilityFile -Path 'C:\RoleCapabilities\helpdesk.psrc'
notepad 'C:\RoleCapabilities\helpdesk.psrc'
```

New PSRoleCapabilityFile Path 'C:\RoleCapabilities\helpdesk.psrc' и го отваряме в notepad

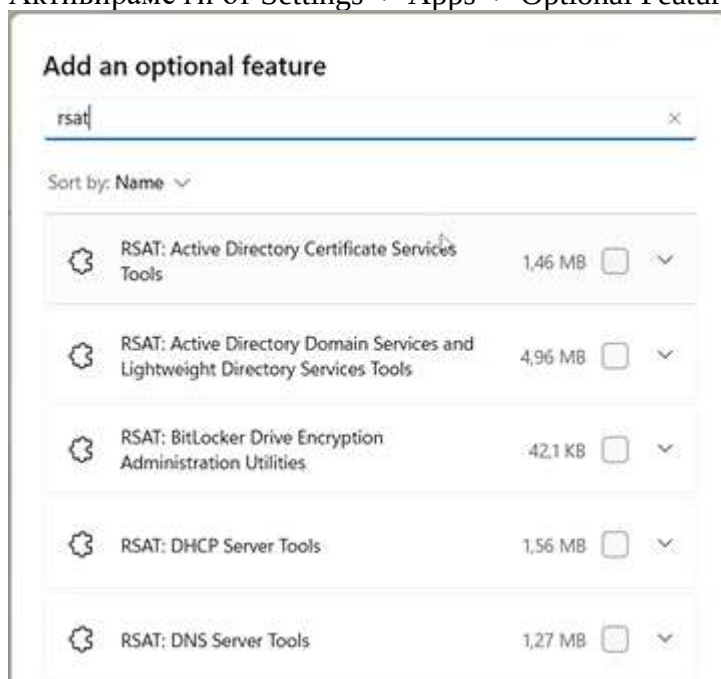
Ctrl + F -> VisibleExternalCommands, изтриваме # за реда, изтриваме Item1,2 и на тяхно място изписваме 'C:\Windows\System32\cmd.exe', като по този начин, тази група има възможност да стартира само външни external commandlets и всичко останало става забранено за тях. След това го активираме чрез Register -PSSessionConfiguration и рестарт на WindowsRemoteManagement service.

```
VisibleExternalCommands - 'Item1', 'Item2'
```

Administiring Active Directory Using PS: Run As Administrator ->

Import-Module ActiveDirectory

Активираме ги от Settings -> Apps -> Optional Features -> Add -> RSAT tools + Ligtweight +



или от Add -WindowsCapabilityCommandlet -Name + името на функционалността.

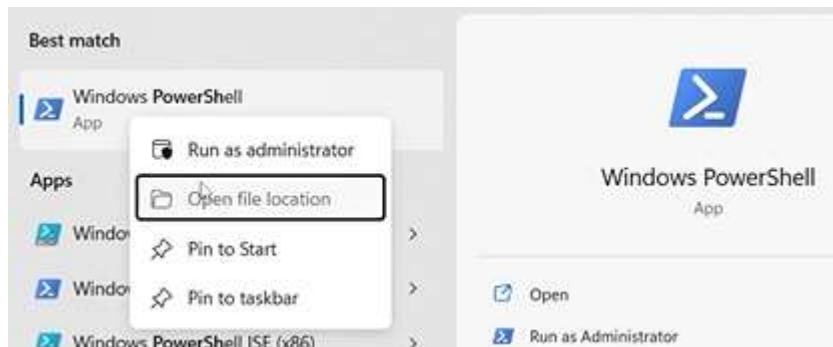
Има 147 ActiveDirectory module commandlets

(Get-Command -Module ActiveDirectory).Count - преброява командите в модула. Без ().Count ги изброява.

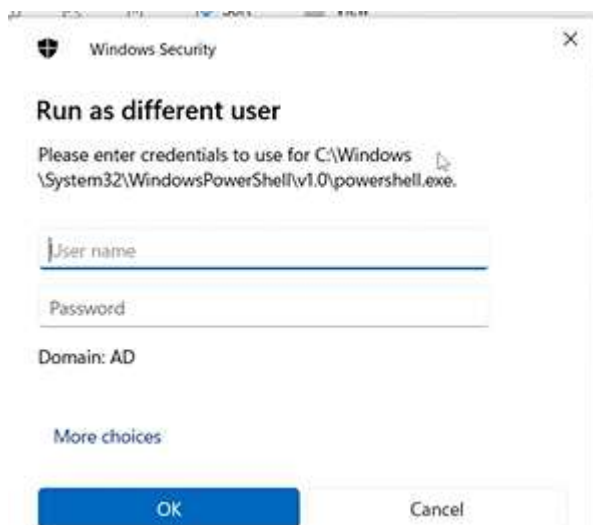
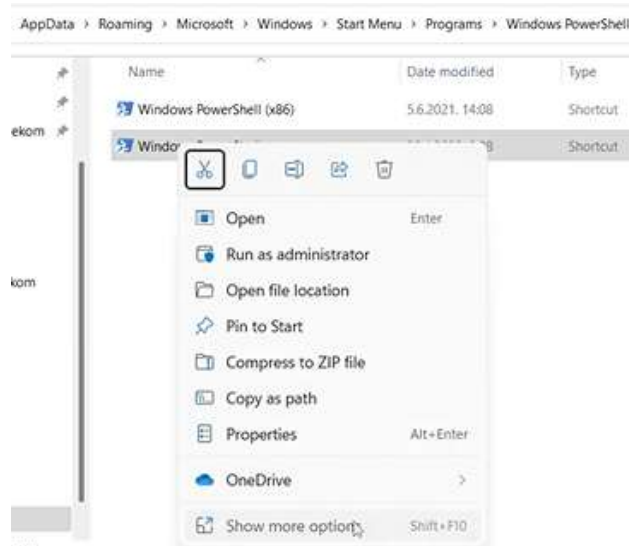
```
PS C:\Users\jsimonec1> Import-Module ActiveDirectory
PS C:\Users\jsimonec1> (Get-Command -Module ActiveDirectory).Count
147
```

Ако искаме повече информация за конкретна команда:  
Get-AdDomain

Можем да създадем нов потребител както на отделния компютър, така и дистанционно чрез new-aduser -name име



Shift + д.б. -> Show more options -> Run as different user



```
PS C:\Users\jsimunc1> New-ADUser
cmdlet New-ADUser at command pipeline position 1
Supply values for the following parameters:
Name: courseuser
```

Добавихме име, но не и парола за новият потребител – това е от особеностите на ActiveDirectory. Нека го изтрием:

```
PS C:\Users\jsimunecl> Remove-ADUser  
cmdlet Remove-ADUser at command pipeline position 1  
Supply values for the following parameters:  
Identity: courseuser  
  
Confirm  
Are you sure you want to perform this action?  
Performing the operation "Remove" on target "CN=courseuser,CN=Users,DC=ad,DC=local".  
[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "Y"): _
```

Можем да добавим и параметри на потребителя като атрибути, например, surname, department, country... Могат да се направят и много потребители наведнъж – bulk user accounts – трябва да имаме предварително подготвена информация за потребителите в .csv формат, вкарана в променлива. Променливата се извика от new-aduser и нейните параметри. Има и достъпни скриптове, които го правят. Могат да се променят и параметрите на конкретен потребител с –enable –adaccount или –disabel –addaccount.

Създаването на компютърен акаунт е по-трудно от създаването на потребителски акаунт, защото админа трябва първо да присъедини компютърните акаунти към домейн и тогава акаунтите се създават автоматично. След това админа премества компютърния акаунт от контейнера по подразбиране към желаната организационна общност. Това работи за малки организации, но за големи се използва New-AdComputer –name и Set-AdComputer за смяна на настройките, new-adGroup – и adGroup –member, new-GPO = group policy, new-GPLink добавя линк към организации, а организации могат да се създават с new-AdOrganizationalUnit.

Powershell Multithreading - нова функционалност, позволяваща стартиране на повече от една команда наведнъж, чрез асинхрона система, сложна и много бърза за голям брой машини, но и изисква повече ресурси.

New-AdGroup – една от най-често използваните команди. Ако искаме поединично да създадем групи за 500 компютъра, по 2 секунди за всеки, това са 1000 сек. Ако обаче използваме multithreading, стартираме 50 и те разпръскват още 10, което ще скъси процеса. Така се подобрява и ускорява кода, а и се избягват излишни цикли.

Get-Command \*-Job – имаме и stop job, remove job, completed job, failed job – на заден план работят.

```
PS C:\Users\jsimunecl> Get-Command *-Job
```

| CommandType | Name        | Version | Source                    |
|-------------|-------------|---------|---------------------------|
| Cmdlet      | Debug-Job   | 3.0.0.0 | Microsoft.PowerShell.Core |
| Cmdlet      | Get-Job     | 3.0.0.0 | Microsoft.PowerShell.Core |
| Cmdlet      | Receive-Job | 3.0.0.0 | Microsoft.PowerShell.Core |
| Cmdlet      | Remove-Job  | 3.0.0.0 | Microsoft.PowerShell.Core |
| Cmdlet      | Resume-Job  | 3.0.0.0 | Microsoft.PowerShell.Core |
| Cmdlet      | Start-Job   | 3.0.0.0 | Microsoft.PowerShell.Core |
| Cmdlet      | Stop-Job    | 3.0.0.0 | Microsoft.PowerShell.Core |
| Cmdlet      | Suspend-Job | 3.0.0.0 | Microsoft.PowerShell.Core |
| Cmdlet      | Wait-Job    | 3.0.0.0 | Microsoft.PowerShell.Core |

Start-Job -ScriptBlock (hostname) – Вместо hostname може да се подаде сложен скрипт, който засегне голям брой компютри. Иначе трябва многократно и поединично да подаваме Start-Job и проверяваме с Get.

```
PS C:\Users\jsimunc1> Start-Job -ScriptBlock { hostname }

Id      Name      PSJobTypeName State      HasMoreData Location      Command
--      -
1       Job1      BackgroundJob  Running    True         localhost     hostname
```

Ipconfig – можем да видим също кои са background jobs, дали работят или са спрени

Get-Job -Id 1 – Проверяваме конкретен

```
PS C:\Users\jsimunc1> Get-Job -Id 1

Id      Name      PSJobTypeName State      HasMoreData Location      Command
--      -
1       Job1      BackgroundJob  Completed  True         localhost     hostname
```

| Receive-Job – връща нашата машина в случая.

```
PS C:\Users\jsimunc1> Get-Job -Id 1 | Receive-Job
WPAK8216
```

## Exercise 01: Apply and Verify DSC Configuration with PowerShell

\* **Get-DscResource** will retrieve the PowerShell DSC resources present in the system. Type Get-DscResource and press **Enter**. DSC resources are the properties that can be configured on the target machine. In the output, you can see **File** at the top of the list, which means files and folders can be managed on the system. You also have **Archive** resource which means archives can be unzipped in .zip format at a particular path. You also have **Environment** resource which means system environment variables can be managed.

\* To check the configuration of the local machine, type Get-DscLocalConfigurationManager and press **Enter**. In the result, you can see **AllowModuleOverWrite** value is set to **False**. It means modules present in the system cannot be overwritten by new modules. The **ConfigurationMode** value is **ApplyAndMonitor** which means configuration is going to be applied and monitored.

\* Type, **notepad** and click on **Notepad**.

```
Configuration randomtext {
  Import-DscResource -ModuleName PsDesiredStateConfiguration
  Node 'localhost' {
    File randomtext {
      DestinationPath = "C:\randomtext.txt"
      Ensure = "Present"
      Contents = "this is some random text"
    }
  }
}
```

It is the configuration script that is going to create a file named **randomtext.txt** with the text value **This is some random text**. -> Save

\* Create New Folder **Scripts** in C:\ Type **dsc.ps1** as **File name** and click on **Save**. This file will be created in a C drive location on a targeted machine. Here we will consider localhost as a targeted machine.

\* C drive is the directory where the configuration file exists. Thus to change the current location, type **cd ..** and press **Enter**. Run the command two times. To list the folders of the current location, type **ls** and press **Enter**. Now, you must change the current location to the script folder. Type **cd script** and press **Enter**. Type **ls** and press **Enter**. You can see the **dsc.ps1** configuration file that is needed.

\* Type **clear** and press **Enter** to clear the screen. To check the status of execution, type **get-executionpolicy** and press **Enter**. **get-executionpolicy** is used to get the execution policy for the current session. You can see it is restricted.

\* Type **set-executionpolicy remotesigned** and press **Enter** to set the execution permission. Type **Y** and press **Enter** when the confirmation for change in execution policy appears. **set-executionpolicy** is used to change the user preferences to execute Windows PowerShell policy.

\* To check the status of execution, type **get-executionpolicy** and press **Enter**. You can see it is **RemoteSigned**.

\* Type **cls** and press **Enter** to clear the screen. To configure the file on the targeted machine, it must be compiled into a mof file. Type **. c:\script\dsc.ps1** and press **Enter** to execute the **dsc.ps1** file.

\* Type **randomtext** and press **Enter**. You can see the **Localhost.mof** is generated in a **randomtext** file. Type **ls** and press **Enter**. You can see the **randomtext** folder in the list which has files in it. The configuration must be applied to those files on a targetted machine.

\* Type **enable-psremoting** and press **Enter**. It enables the PSRemoting to connect to the machine.

\* Type **Start-DscConfiguration -path C:\script\randomtext -verbose -Wait** and press **Enter**. Thus, the DSC is applied to the targeted machine.

\* To verify it, you must open the **randomtext.txt** file. Minimize the **Windows PowerShell CLI**. Click on the **File Explorer** icon. Under **Quick access**, click on **This PC**. Double-click on **randomtext.txt** to open it. You can see the **randomtext.txt -Notepad** appears with the content as **this is some random text**. This concludes the demonstration of **Apply and Verify DSC Configuration with PowerShell**.

## Exercise 02: Configure and Customize PowerShell Session and Role Capabilities

\* Type **enable-psremoting** and press **Enter**. It enables the PSRemoting to connect to the machine.

\* Type **get-executionpolicy** and press **Enter**. It displays **RemoteSigned**. Run the commands mentioned below if you have not set the execution policy.

**set-executionpolicy remotesigned**

**get-executionpolicy**

\* Type **Get-PSSessionConfiguration** and press **Enter**. It gives the session configurations registered on the local machine. A session configuration file is one in which the specified setting on the local machine is going to define the environment for the upcoming PowerShell sessions that will be created when other users connect to the PowerShell on that particular machine. That configuration file will be specific for a specific machine.

\* C -> new folder name as **jea** and press **Enter**. **jea** is the folder in which you will save the

session configuration file via the PowerShell cmdlet.

- \* To create your own configuration file, type `New-PSSessionConfigurationFile -path 'C:\jea\jeaconffile.pssc'` and press **Enter**. **jeaconffile.pssc** is the name of the new configuration file that will be created.

- \* Minimize the **Windows PowerShell** window. Navigate back to the **File Explorer** from the taskbar. Double-click on the **jea** folder in which the file is saved via cmdlet.

- \* You can see a text file **jeaconffile.pssc** is saved. Double-click on it to open. For **how do you want to open this file?** Select on **Notepad** and click on **OK**.

- \* The script will open in a **Notepad** as shown. In the script, anything written after **#** will be considered as comment.

- \* You can make changes according to your needs, concerning user privilege. Change the **SessionType** assigned value from **Default** to **Restricted RemoteServer**. The above-made changes will provide the ability to execute a few key cmdlets.

- \* Assume that you have an active directory group named **helpdesk** which contains users that will need to apply this configuration. Change the **RoleCapabilities** assigned value from **SqlAdministrattion** to **helpdesk**.

- \* Press **Ctrl+S** to save the changes. Then click on the **Close** icon to close it.

- \* Now, you have to create a folder in **C:** drive to save roles that will be detected by PowerShell. Click on the **back arrow** in the **File Explorer** to back by one step. Right-click on the page. Under **New** select **Folder**. Type the folder name as **RoleCapabilities** and press **Enter**.

- \* **.psrc** extension file has to be created to configure the members of the helpdesk group in the active directory. Go back to the **Windows PowerShell** from taskbar. Type `New-PSRoleCapabilityFile -path 'C:\RoleCapabilities\helpdesk.psrc'` and press **Enter**.

- \* Minimize the **Windows PowerShell**. In the **File Explorer** window double-click on the **RoleCapabilities** folder to open it. You can see a file is saved. Double-click to open it. For **How do you want to open the file?**, click on **More apps**. Select **Notepad** and press **OK**.

- \* The script text file opens as shown. The changes that will be made in it will restrict the members of the local directory group helpdesk in such a way that they will be able to run only certain cmdlets.

- \* Find the line **#VisibleExternaleCommands = 'Item1','Item2'** to change its assigned value. Remove the **#** and assign the value as given below. According to the changes, members of the help group will be able to execute only the external command named **cmd.exe**.

- \* This way you can whitelist or blacklist the cmdlets for the users.

## Exercise 03: Implement and Manage Active Directory User Accounts using PowerShell

- \* Run the command `Import-Module ActiveDirectory` to import Active Directory.

- \* Run the command `(Get-Command -Module ActiveDirectory).Count`. This will give you the total number of commands.

- \* Run the command `Get-Command -Module ActiveDirectory`. This will display the list of commands.

- \* To create a new account in the active directory, run the command **New-ADUser**. It will ask for a name, type **courseuser**, and press **Enter**.

- \* Now to remove this account, run the command **Remove-ADUser**.

- \* It will ask for **Identity**, type **courseuser** and press **Enter**.

\* It will ask for confirmation, press the **Y** key to continue. The account will be removed from the active directory.

\* This concludes the demonstration of **Implement and Manage Active Directory User Accounts using PowerShell**.

## Exercise 04: Analyze and Manage Background Jobs in PowerShell

\* Type the below command and press **Enter**. You will get a list of cmdlets that end with **-Job**.

Get-Command \*-job

Specific cmdlet is used for specific functions. **Start-Job** is used to start the background job, **Get-Job** is used to get the list of background jobs running, and **Remove-Job** will delete the background job. And the background jobs can be in various states which define how PowerShell is handling and defining the job.

\* Type **Start-Job -ScriptBlock { hostname }** and press **Enter**. You can see in the result that the PS Job type is **BackgroundJob** and the **State** is **Running**. **Start-Job** will start a background job on a machine. **ScriptBlock** is a parameter of **Start-Job** cmdlet. In the place of **hostname** one will enter the script that has to be run in the background so that other cmdlets can be executed in the foreground.

\* Once you start a background job, try to run a cmdlet in the foreground. Type **ipconfig** and press **Enter**. You can see that the command is executed.

\* To check the status of the background job, type **Get-Job -id 1** and press **Enter**. You can see it is completed. **1** was the ID that the system assigned to the background job.

\* As the background job is completed, you can check the result. Type **Get-Job -id 1 | Receive-Job** and press **Enter**.

\* One can run complicated scripts simultaneously in the background. Type **Start-Job -ScriptBlock { hostname }** and press **Enter**. A new background job will get started. Practically, a different script will be written instead of **hostname**.

\* To run another background job, type **Start-Job -ScriptBlock { hostname }** and press **Enter**. A new background job will get started. Running multiple background jobs at once leads to an increase in the use of system resources.

\* To get the list of backgrounds, type **Get-Job** and press **Enter**. **Get-Job** command will list all the background jobs in the present session.

\* This concludes the demonstration of **Analyze and Manage Background Jobs in PowerShell**.

Edit group policy

- Group policy module installed

Get-Command -Module [име по избор]

```
PS C:\Users\jsimunecl> Get-Command -Module GroupPolicy
```

| CommandType | Name                    | Version | Source      |
|-------------|-------------------------|---------|-------------|
| Alias       | Get-GPPermissions       | 1.0.0.0 | GroupPolicy |
| Alias       | Set-GPPermissions       | 1.0.0.0 | GroupPolicy |
| Cmdlet      | Backup-GPO              | 1.0.0.0 | GroupPolicy |
| Cmdlet      | Copy-GPO                | 1.0.0.0 | GroupPolicy |
| Cmdlet      | Get-GPInheritance       | 1.0.0.0 | GroupPolicy |
| Cmdlet      | Get-GPO                 | 1.0.0.0 | GroupPolicy |
| Cmdlet      | Get-GPOReport           | 1.0.0.0 | GroupPolicy |
| Cmdlet      | Get-GPPermission        | 1.0.0.0 | GroupPolicy |
| Cmdlet      | Get-GPPrefRegistryValue | 1.0.0.0 | GroupPolicy |
| Cmdlet      | Get-GPRegistryValue     | 1.0.0.0 | GroupPolicy |

Виждаме New-GPO = new group policy object  
 New-GPLink = new existing group policy link  
 Get-GPResultantSetOfPolicy – изкарва рапорт за политиките

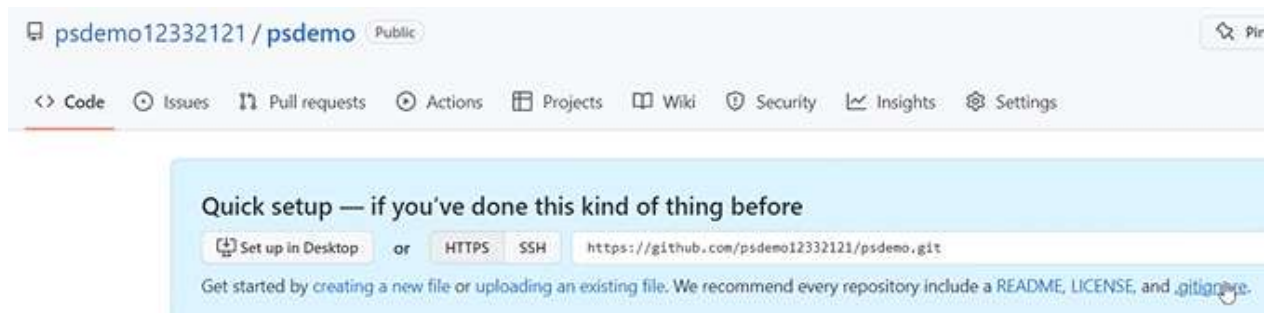
Creating our own PS Commandlet -

Модулите са групи от команди .psm1, а стандартните скриптове .ps1

Error Handling –

Можем да си държим кода локално на машината, а може и да го споделим в GitHub, където може да проследяваме с commit. Създаваме публичен акаунт, безплатен, за студенти, без лиценз, но никой не би използвал нашия код, ако няма лиценз, така че е хубаво да слагаме.

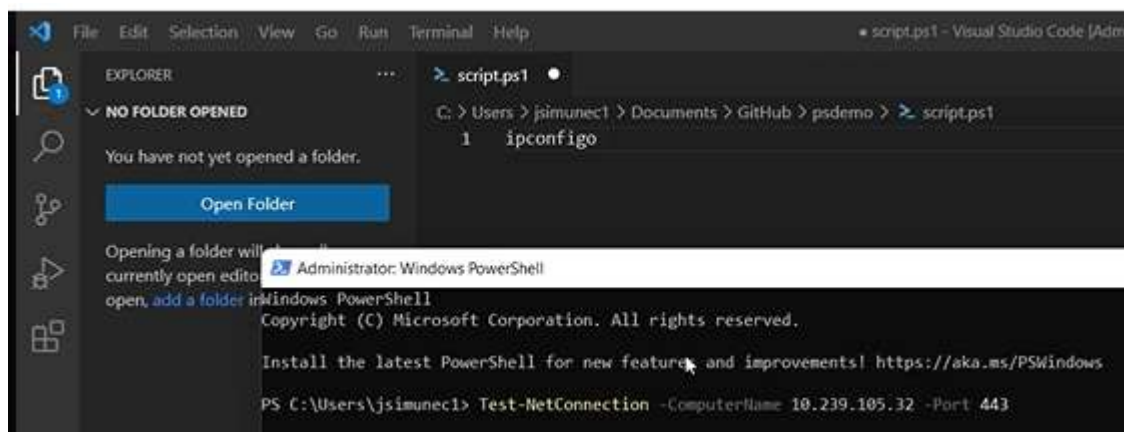
Имаме инструменти и долу вдясно – gitignore



Github desktop download е синхронизиран с браузernия – клониране = изтегляне локално

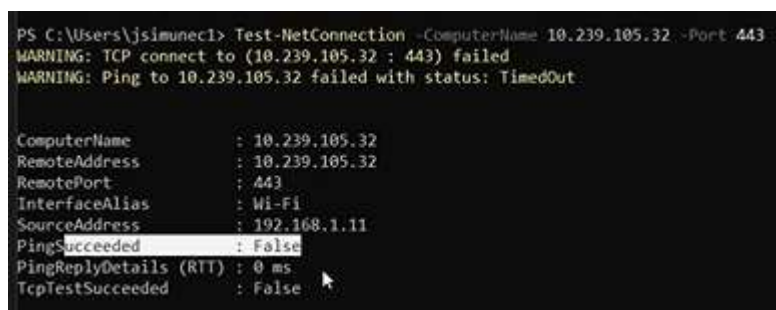
Създаваме файл, избираме го в десктоп версията -> repository -> pull

За редакция д.б. -> Open with default program, след редакция changes, history, update...push to origin

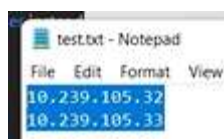


Test-NetConnection –ComputerName 10.239.105.32 –Port 443

Просто тест, свързването е неуспешно



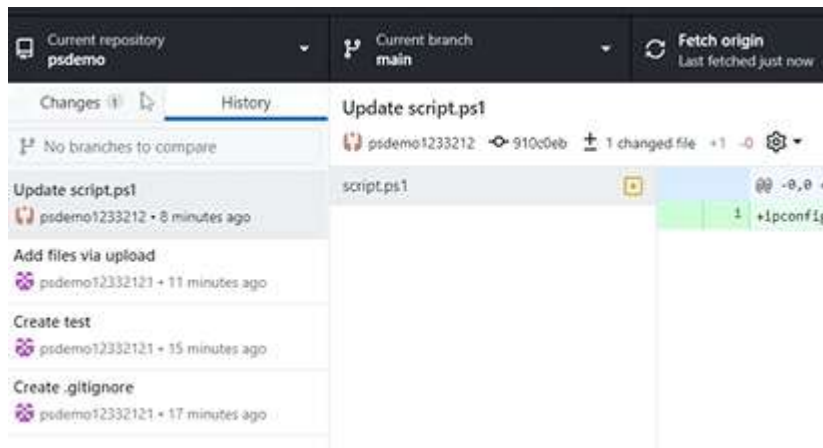
Създаваме файл test.txt с няколко ip адреса за по-бързо



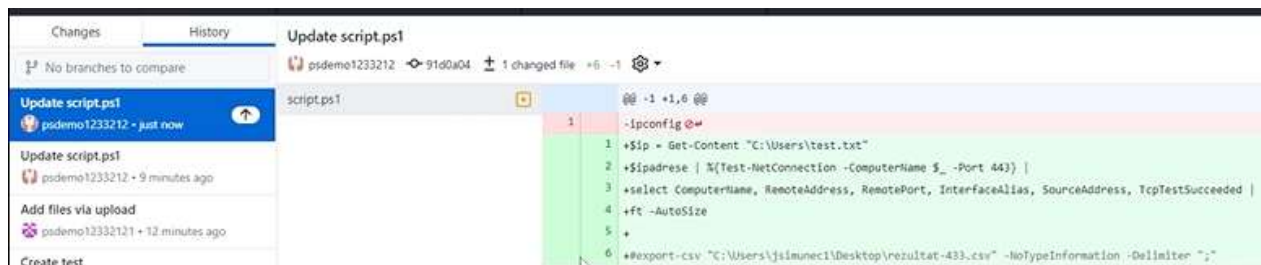
\$ = променлива



Извличаме адресите от файла – запазваме файла, после



Fetch origin, commit changes to main -> history



Резултатът трябва да се изведе в конкретен файл на десктопа.

PS logging, auditing and recording

Logging with transcripts = най-простият метод за вписване: извличат се всички commandlets и изходи, които се генерират по време на сесия. При затварянето на сесията, се затваря и транскриптът. За да се включи подобна транскрипция, може да се редактира регистъра ръчно за 1 машина или за огромен брой машини чрез group policy. При старт се правят записвания capture – показва ни се къде и името на файла, а при stop се вкарват в регистъра.

Start-Transcript

Stop-Transcript

```
PS C:\Users\jsimunc1> Start-Transcript
Transcript started, output file is C:\Users\jsimunc1\Documents\PowerShell_transcript.WMAK8216.AqBoubZb.20230202074632.txt
PS C:\Users\jsimunc1>
```

Между start – stop се записват команди и действия, които искаме. Ако напишем Get-Command, ще запише хилядите команди. Ако запишем hostname – името на компютъра.

```
PS C:\Users\jsimunc1> Stop-Transcript
Transcript stopped, output file is C:\Users\jsimunc1\Documents\PowerShell_transcript.WMAK8216.AqBoubZb.20230202074632.txt
```

## Exercise 01: Analyze and Implement Group Policy Management Using PowerShell Cmdlets

- \* Firstly, you must change the current location to **C:\Users\Admin**. So type **cd ..** and press **Enter**. Run the command twice as shown in the image. Type **cd Users** and press **Enter** to change the current location to **C:\Users**. Type **cd Admin** and press **Enter** to change the current location to **C:\Users\Admin**. Thus, you have reached the required location.
- \* Type **Get-Command -Module GroupPolicy** and press **Enter**. You might not get the result of a list of cmdlets for GroupPolicy if the group policies are not configured on the system.
- \* You can see the list of cmdlet information in Google. Minimize the **Windows PowerShell**. Click on the **Google Chrome** icon on the desktop. Type **GroupPolicy** module in the URL search bar and press **Enter**. Click on the official link of **Microsoft GroupPolicy Module**. It is highlighted in the image.
- \* In the list of GroupPolicy cmdlets, click on **New-GPO**. The **New-GPO** detailed information will be shown. Scroll down and read the information given about **New-GPO**. [!note] The **New-GPO** command is used to create a completely new group policy object. The **New-GPOLink** command can be used to link the existing group policy object to the targeted organizational unit.
- \* On the left side of the page you can see the list of cmdlets. Click on **Get-GPO**. It gives information about one or all GPOs of the target.
- \* In the list of cmdlets on the left side of the page, click on **Remove-GPO**.
- \* In the list of cmdlets on the left side of the page, click on **Invoke-GPUUpdate**. The cmdlet is used to update group policy settings on the targeted remote machine or all the machines in the organizational unit. The information about the **Invoke-GPUUpdate** cmdlet will be shown.

## Exercise 02: Analyze and Implement Script Version Control with GitHub and PowerShell

- \* The **Administrator: Windows PowerShell ISE** appears as shown below. Click on **New Script** icon. A new PowerShell script page appears as shown in the image.
- \* Other than the local machine, one can also save the scripts in GitHub. For that firstly, you must create a GitHub account. Minimize the **Windows PowerShell ISE**. Click on the **Google Chrome** icon on the desktop.
- \* The **Welcome to GitHub** page appears. For **How many team members will be working with you?** select **Just me**. Select **Student**. Then click on **Continue**. Then **What specific features are you interested in using?** option appears. Scroll down and click on **Continue**. The GitHub recommended plans option appears. As you need free GitHub, scroll down and click on **Continue for free**. Your GitHub account dashboard appears.
- \* To **Start a new repository** type the repository name and make it **Public**. Then click on **Create a new repository**. Here the new repository name given is **jcdemo**. GitHub contains public and private repositories. One can keep the code by oneself in a private repository or can keep it in a public repository to share it with other people and receive reviews for your code from other people. The newly created repository opens as shown below. By default, the **Code** section will be opened. You can see a list of tabs available - Code, Issues, Pull requests, Actions, Projects, Wiki,

Security, Insights, and Settings.

- \* Click on **gitignore**. The **gitignore** file appears, where you can type the content of your wish.

- \* Type **test** and click on **Commit changes**. Again under the pop-up, click on **Commit changes**.

- \* You can see **.gitignore** file becomes the branch under main **jcdemo** repository. You can add multiple branches of your wish.

- \* You can add a code file to your repository. Click on **Add file**. Then select **Create a new file**.

- \* In the **Name your file** input section type **test** as the file name. Type **hostname** as the content of the file. Then click on **Commit changes**. Here **hostname** is the cmdlet. Instead of typing long code, you will be using this cmdlet as a simple code for demonstration. For the pop-up, again click on **Commit changes**. You can see the **test** file has been added to your repository.

- \* Now, you will be downloading the GitHub desktop version. Click on the + icon to open a new tab. Once the new tab opens, type github desktop download in the search bar and press **Enter**. To authorize the GitHub Desktop web browser will open automatically. Click on **Authorize desktop**. The **Configure Git** appears. Check the **Name** and **Email**. Click on **Finish**.

- \* The add repository option appears. Under **Your repositories**, you will get the name of the repository that you created. Select the repository and click on **Clone**. For the **Clone a repository** pop-up, click on **Clone**. Thus the GitHub Desktop repository is created successfully.

- \* Now, you will learn how to add a script file from the local host to the GitHub account.

Minimize the **GitHub Desktop** and the browser. Click on **File Explorer**. Firstly, you will create a folder in which the script text document will be saved. Select **This PC**, then double click on **Local Disk (C:)**. Right-click and select **New**. Then click on **Folder**. Type **script** as a folder name and press **Enter**. Minimize the **File Explorer**. Click on the **search** icon. Type **notepad**. Click on **Notepad**. The notepad file appears as shown. Press **Ctrl+S** to save the empty demo file. Select **This PC** and double click on **Local Disk (C:)**.

- \* Double-click on **script**. Type **File name** as **script.ps1** and click on **Save**.

- \* Click on your GitHub repository tab as shown. Click on **Add file** and select **Upload files**.

Select **This PC** and double click on **Local Disk (C:)**, **script** folder, **script.ps1** and click on **Open**. You can see that **script.ps1** is uploaded. Scroll down and click on **Commit changes**. You can see in the repository that **script.ps1** is added.

- \* Click on **Repository** in the tool section. Select **Pull**. Wait until the fetching is completed.

- \* Click on the **History** section which is shown in the image. To the left side of the page, you can see the list of actions you performed. Click on **Add files via upload**. You can see the **script.ps1** file to its right side. It is the file that you uploaded. You can see that **The file is empty**.

- \* Minimize **GitHubDesktop**. And re-open the minimized Web Browser. Click on **script.ps1** file in the repository. You can see that it is empty. Thus, you can say that the GitHub repository account is synchronized.

- \* You can also open and edit the files. Right-click on **script.ps1**. Select **Open with default program**. The file will open in **Notepad**. Type **ipconfig** and press **Ctrl+S** to save the content added to the file. Close the **Notepad** and then click on **Changes** section.

- \* The changes made to **script.ps1** is shown. To save the changes made in your repository, click on **Commit to main**.

- \* Again click on **History** section. Select **Upload script.ps1**.

- \* To your right-hand side, you can see the content that was added to the file. Click on **Push origin** to push the changes made to your GitHub repository.

- \* To check whether the made changes are synchronized in the cloud, minimize **GitHub Desktop** and re-open minimized **Google Chrome**.

- \* Click on **jcdemo** which is the main branch. Then click on **script.ps1**.

- \* You can see that changes were made to the **script.ps1** file content in the **GitHub Desktop** are reflected successfully.
- \* You can also open and edit the script files in other programs like Visual Studio Code. For that firstly, you must download Visual Studio Code. Click on the + icon to open a new tab.
- \* Select **VisualStudioCode** as the application to view the file and click on **OK**. Select **script.ps1**. Right-click on it and select **open with default program**. The file **Script.ps1** will be opened in **VisualStudioCode**. Thus, it is clear that you can open the file in other applications where you can modify the content.
- \* Type **powershell** and click on **Windows PowerShell** to open it.
- \* Type `Test-NetConnection -ComputerName 10.239.105.32 -Port 443` and press **Enter**. You can see in the result that the TCP connection failed. Here, you will test for network connection with a parameter computer name with a certain IP address and for HTTPS port 443.
- \* Once the **Notepad** opens. Type the below IP addresses as the content.  
10.239.105.32  
10.10.1.13
- \* Press **Ctrl+S** to save it. Under **This PC** double click on **Local Disk (C:)**. Double-click on the **script** folder. Type **test.txt** as the file name and click on **Save**.
- \* In VSCode:  
\$ip = Get-Content "C:\script\test.txt"  
\$ip | %{Test-NetConnection -ComputerName \$\_ -Port 443} |  
select ComputerName, RemoteAddress, RemotePort, InterfaceAlias, SourceAddress,  
TcpTestSucceeded |  
ft -AutoSize  
export-csv "C:\Users\Admin\Desktop\rezultat-433.csv" -NoTypeInfoation -Delimiter ";"

In this code, the Ip address in the text.txt file will be read to check for the network connection and display the result. It also created a rezultat-433.csv file.

- \* To save the changes made, click on **File** tab and select **Save**. Now check whether the same changes made in **VisualStudioCode** are reflected in the **GitHub Desktop** repository. Minimize the **VisualStudioCode**. Re-open **GitHub Desktop**.
- \* Select **script.ps1**. Then click on **Fetch origin**.
- \* Then click on **Changes** section. Then click on **Commit to main**.
- \* Then select the **History** section and click on **Update script.ps1**. The file content will be updated as shown.
- \* Now, you will run the same script in the **Windows PowerShell ISE**. Minimize the **GitHub Desktop** and re-open **VisualStudioCode**. Select the script as shown and press **Ctrl+C** to copy it.
- \* Minimize **VisualStudioCode**. Click on the **Windows PowerShell ISE** icon from the desktop tool. Press **Ctrl+V** to paste the script. Select the content and press **F8**. to run the selected script.

## Exercise 03: Create and Document a PowerShell Session Using Start-Transcript and Stop-Transcript

- \* The **Administrator: Windows PowerShell** appears as shown below. Firstly, you must change the current location to **C:\Users\Admin**. So type `cd ..` and press **Enter**. Run the command twice as shown in the image. Type `cd Users` and press **Enter** to change the current location to

**C:\Users.** Type **cd Admin** and press **Enter** to change the current location to **C:\Users\Admin.** Thus, you have reached the required location. Type **hostname** and press **Enter.** It gives the name of your host.

\* Minimize the **File Explorer** and click on **Windows PowerShell** icon again to re-open it. Type **Start-Transcript** and press **Enter.** **Start-Transcript** will create a record of the powershell session to a text file.

\* Type **hostname** and press **Enter.** Type **Get-Command** and press **Enter.** The result will be the list of commands of PowerShell that are installed on the system.

\* Type **Stop-Transcript** and press **Enter.** Minimize the **Windows PowerShell.** Click on **File-Explorer** to re-open it. You can see a text file is saved in the **Documents** folder. Double-click on it to open the text file. You can see the entire session between **Start-Transcript** and **Stop-Transcript** is saved in the file.

.....

It is mandatory to understand  
You need to consider  
Nevertheless – въпреки това  
It's big advantage  
Simultaneous – едновременно  
Intended – предназначен  
We are going to choose the first page that pops out  
I will erase this command  
Whatever version suits your machine  
Couple of more seconds  
The download is over  
This installation is straightforward  
Plus = + плАс  
Commandlet auto suggestion  
It is time to dive in  
Invoke – призовавам  
Fetch – донасям  
Occupied – зает  
Our screen is occupied with this very long list  
Proficient – умел, опитен  
To be proficient in navigating in  
Besides – освен това  
Beside – до, покрай  
In certain matters  
[] – square brackets  
Widely used  
That should be known to everyone dealing with  
Allotted – разпределен  
As we have mentioned before  
This is very beneficial for  
Granting – предоставяне  
Examination – преглед  
Repetitive – повтарящ се  
Severity – тежест  
Let's close quotations  
If you try to... you would not be able to do so  
You could notice the error  
Let's confirm that – нека го потвърдим/докажем  
As you can conclude

Toggle – превключвател  
In order to see...  
Visually behaved similarly  
Deploy - разположи  
Enact – приемам  
Verbose – многословен  
As it is usually for most of the things –  
Prerequisite – предпоставка  
Enumerate – изброявам  
# = hash, hashes  
Embedded comment  
Will be regarded – ще бъде считан  
Deployment – внедряване  
Bulk – насипно състояние  
Threading – резба  
MultiThreading – многонишково четене  
Adjust – коригирам  
Foreground – преден план  
In following way – по следния начин  
Inside the brackets I will put – () – между скобите  
Simultaneously - едновременно  
Anomaly – аномалии  
deviation – отклонение  
derive – извличам – дерАйв  
derived – извлечен  
derivative – производно – деривЪтив  
Commit – ангажирам, поверявам, въвличам  
Error handling – Обработка на грешки  
handling – боравене  
Contribute – допринасям, сътруднича  
Which I have previously created on my C drive  
It may execute very slowly depending on bandwidth and other network parameters



